DEBRE TABOR UNIVERSITY

GAFAT INSTITUTE OF TECHNOLOGY

DEPARTMENT OF INFORMATION TECHNOLOGY

Project Title: - Web-Based E-Bid system

A project submitted in partial fulfillment of the Bachelor of Science in Information Technology.

Prepared by:

Student's Name Id

1. Amarech Guadie …………………………………………… 0513
2. Shishig Desalegn……………………………………………1439
3. G/Mariam Abirham………………………………………… 1094
4. Meseret Andargie……………………………………………0917
5. Aklilu Ebakum……………………………………………… 0327

Project Advisor: **Mr. Habtamu Kasa**

2026 Debre Tabor
Ethiopia

PROJECT APPROVAL FORM

Program:

this project, titled **"E-Bid Processing System"**, is submitted by the following students for the partial fulfillment of the requirements for the Degree of Bachelor of Science in **Information Technology** at **Debre Tabor University, Institute of Technology.**

Student Name	**Signature**	**Date**
AMARECH GUADIE	____________________	____________
SHISHIG DESALEGN	____________________	____________
G/MARIAM ABIRHAM	____________________	____________
MESERET ANDARGIE	____________________	____________
AKLILU EBAKUM	____________________	____________

Approved By Board of Examiners:

The project has been reviewed and approved for submission by the following board of examiners:

1. Project Advisor Name: **Mr. Habtamu Kasa.** Signature: ____________________ Date: ____________

2. Department Head / Coordinator Name: _______________________ Signature: ____________________ Date: ____________

3. Internal Examiner Name: _______________________ Signature: ____________________ Date: ____________

4. External Examiner Name: _______________________ Signature: ____________________ Date: ___________

ACKNOWLEDGEMENTS

First and foremost, we would like to express our deepest gratitude to the Almighty **God** for His guidance, strength, and blessings throughout our journey. His presence provided us with the wisdom and perseverance needed to complete this project successfully.

Secondly, we would like to extend our sincere appreciation to our advisor, **Instructor Mr. .**, for his invaluable guidance, professional comments, and continuous support. His expertise and constructive advice played a significant role in shaping the direction of this project and helping us meet our objectives.

ABSTRACT

The current procurement and bidding process at **Debre Tabor University (DTU) Institute of Technology (IOT)** is primarily a manual, paper-based system. This manual workflow is often characterized by significant manpower consumption, material wastage, lack of transparency, and delays in communication between departments and suppliers. To address these challenges, this project focuses on the design and development of a web-based **E-Bid Processing System**.

The proposed system is built using the **MERN stack**, which comprises **MongoDB, Express.js, React.js, and Node.js**. This modern JavaScript-based architecture provides a high-performance, scalable, and secure digital environment. Key functionalities of the system include electronic purchase requisitions, automated inventory and budget verification, digital tender announcements, secure online supplier registration, and electronic bid submissions.

Furthermore, the system implements robust security measures, including **JSON Web Tokens (JWT)** for authentication and data encryption to protect sensitive financial information until the official bid opening. By automating the procurement lifecycle, the system aims to improve administrative efficiency, reduce human error, and foster a fair and competitive bidding environment for all participating vendors. Ultimately, this project transitions DTU IOT's procurement activities into a modernized, transparent, and cost-effective digital platform.

ACRONYM	DEFINITION
API:	Application Programming Interface.
ERD:	Entity Relationship Diagram.
ES6+:	ECMAScript 6+ (Modern JavaScript).
HTTP:	Hypertext Transfer Protocol.
HTTPS:	Hypertext Transfer Protocol Secure.
IOT:	Institute of Technology.
JSON:	JavaScript Object Notation.
JWT:	JSON Web Tokens.
MERN:	MongoDB, Express.js, React.js, and Node.js.
NPM:	Node Package Manager.
ODM:	Object Data Modeling.
QA:	Quality Assurance.
REST:	Representational State Transfer.
SMTP:	Simple Mail Transfer Protocol.
SPA:	Single Page Application.
SRS:	Software Requirement Specification.
UI:	User Interface.
UML:	Unified Modeling Language.
UPS:	Uninterruptible Power Supply.
UX:	User Experience.

Table contents

List of tables

List of figures

1. Chapter 1: Introduction

An e-Bid processing system is the process of demand, demand approval, bid in public sale and ship product. It is a system that holds online bid for various items on a website and serves sellers and bidder accordingly. It is responsible for purchasing needed items for all its users. This means the role of the purchasing team is to work as a strategic pattern within the organization to maximize the value of its users. The system is designed to allow users to set up their products/items for auction and to register and bid for various items available for bidding.Usually, bidding has a start time and ending time. A potential supplier in the auction and the winner is the one who passed the technical evaluation and also that supply the products with fair price and with high quality (i.e. that passed technical and financial evaluation).

In order to participate in our bidding system, you must first register. In this system the company/organization post products that the company wants to buy from suppliers.Now a day IT can play a great role for higher educational institutions application such as e-learning, web-based procurement system and etc. Due to these the current technology, the proposed system to overcome some problem done by the manual system of procurement to the computerized system for the users.

The role of web-based for the Bid system is as current technology increases the system willproduce well-designed tasks and the system contains security and authentication. Not only this but also as a procurement system based on the web-based we can easily manage any information performed by this system.

1.1. Background (the organization and the project)

Debre Tabor Universty IOT Bid Processing system is one of the processes that will be done when departments and communities want to buy and sell materials and other things. When we observing the existing purchasing system of Debre Tabor University Institute of technology it works various activities as manually.

The number of users and need of departments increases from year to year in many numbers. So,in order to fulfill the need of the users and departments purchasing team performs different activities.

- Gathering departments demand.
- Analysis of the market prices for each material.
- Select the suppliers that offer reliable products at fair prices.
- Improve the relationship between suppliers and buyers.

1.2. Motivation

The primary motivation for developing this web-based E-Bid Processing system for **Debre Tabor University (DTU) IOT** is to modernize the current manual procurement process, transitioning it into a more efficient, transparent, and secure digital environment. The system aims to achieve the following:

- **Improved Management and Service**: To enhance the overall management and delivery of purchasing services within DTU IOT.
- **Enhanced Documentation**: To allow for easier document management and faster communication between purchasers and suppliers.
- **Reduced Manual Workload**: To overcome the high demand for manual resources and reduce the physical effort required by various departments.
- **Error Reduction**: To minimize the rate of human error inherent in paper-based registration and data entry.
- **Time Efficiency**: To eliminate the time spent by the purchasing team manually entering supplier bid information and to save overall processing and delivery time.
- **Uniformity and Fairness**: To ensure that uniform bid information is relayed to all potential suppliers, fostering a fair, open, and accessible bidding environment.

- **Reliable Approvals**: To provide a faster and more reliable mechanism for the approval of requested items by university leadership.
- **Cost Savings**: To reduce the university's expenditure on manpower and materials, such as paper and printing, which are heavily consumed in the current manual system.
- **Increased Participation**: To increase the number of participating bidders by improving the delivery of bid announcements through a web-based platform.
- **Security and Authorization**: To implement robust security mechanisms, including authentication and authorization processes, to protect sensitive procurement data.

1.3. Statement of the problem

There is no aImprove the relationship between suppliers and buyersutomated system in the purchasing office. Due to this reason, the user request uses

paper to fulfill all information and activities performed in purchasing activities. The problems

that existed in the current system are listed below:

- Consume more manpower and material wastage.
- The tender cannot be performed in a fair way. This means corruption may be available.
- No more bidders are participating due to lack of announcement delivery.
- Consume more time.The system has no security mechanism such as an authentication and authorization process.
- The supplier cannot get information easily since it is a manual system.
- Difficult to manage the user's request.

1.4. Objectives

1.4.1. General Objectives

to design and develop web-based Bid Processing system for Debre Tabor University IOT.

1.4.2. Specific Objectives

In order to achieve the general objective of the system, the following specific measurements and actions will be taken:

- **System Analysis**: To investigate and analyze the current manual procurement and bidding system.
- **Requirement Identification**: To determine the functional and non-functional requirements necessary for the new automated system.
- **System Modeling**: To construct use case diagrams and other models that accurately describe the interaction between users and the system.
- **User Interface Design**: To design a user-friendly and interactive interface for the new web-based system.
- **Access Control**: To create and implement different access roles and privileges for various users within the system.
- **Web Development**: To develop the website according to the specific business requirements of the university.
- **Database Design**: To design an efficient database to securely store and maintain all records associated with the bidding process.
- **Database Implementation**: To develop and deploy the database for the system's recording mechanism.
- **System Testing**: To conduct thorough testing of the system to ensure all functions operate correctly and overcome potential errors.
- **Reporting**: To enable the system to generate various reports as demanded by the bid processing requirements.

1.5. Scope of the project

The scope of this project involves the implementation and development of a web-based system capable of providing and managing relevant information for the **Debre Tabor University (DTU) IOT** purchasing system. While the overall procurement volume in such an organization is wide, this proposed system will automate the partial services of the organization related to bid processing.

The system specifically supports the following Bid Processing activities:

- **Request Approval**: Facilitating the approval of user requests.
- **Purchasing Formats**: Enabling users to view standard purchasing formats.
- **Bidder Evaluation**: Providing tools to evaluate documentation submitted by bidder.
- **Inventory Management**: Registering available data and items currently in the store.
- **Auction Orders**: Allowing authorized users to order purchasing auctions.
- **Budget Oversight**: Approving or checking available budgets for procurement.
- **Communication**: Enabling users to send and receive messages within the system.
- **Winner Notification**: Notifying the successful bid winner.
- **Tender Management**: Enabling the upload of tender formats and documentation.
- **Requisition Arrangement**: Arranging requisitions by balancing requested materials against available stock and budget.
- **Data Security**: Enabling the encryption of sensitive supplier information.

1.6. Significance of the Project

The primary aim of the proposed web-based system is to enhance the overall management and services of the purchasing system for **Debre Tabor University IOT**. The core significance of this project includes:

- **Efficient Document Management**: Purchasers can manage documents more easily through quick and simplified communication channels.
- **Reduced Resource Demand**: The system helps overcome the high demand for manual resources within the department.
- **Error Reduction**: Implementing the system will lead to a reduction in manual error rates during procurement activities.

- **Automated Data Entry**: It eliminates the time previously spent by the purchasing team manually entering supplier bid information.
- **Uniformity of Information**: The system ensures that uniform bid information is relayed to all potential suppliers, fostering transparency.
- **Time Efficiency**: It significantly saves processing time and improves overall delivery times for procurement services.
- **Reliable Approvals**: It provides a fast and reliable mechanism for the approval of requested items.
- **Fairness and Accessibility**: The entire bidding process becomes fair, open, and easily accessible to all vendors and suppliers.

1.7. Feasibility Study

The feasibility study for the E-Bid Processing system at **Debre Tabor University (DTU) IOT** evaluates the project's viability across four main areas: technical, operational, economic, and schedule feasibility.

To ensure user acceptance and system clarity, the following feasibility studies were conducted:

1.7.1. Technical Feasibility:

This study confirms that the new system can be implemented using current technology. It is designed to be user-friendly, meaning it does not require professional IT staff for daily use, allowing users both inside and outside the university to access it without confusion.

1.7.2. Operational Feasibility:

This measures how effectively the proposed system solves existing problems. The project is deemed operationally feasible as it provides a robust solution to current manual processing issues and creates a favorable environment for all users.

1.7.3. Economic Feasibility: This is a measure of cost-effectiveness, identifying the benefits relative to the costs.

Table 1.1 Economic cost estimation

No	Material	Amount	Price for Each (ETB)	Total Price (ETB)
1	Paper	Up to 500	1.5	750
2	Pen	5	30	150
5	Ruler	1	6	6
6	Copy and print	300	1.5	450
7	Removable flash (8GB)	1	350	350
8	Laptop	2	--	--
10	Software	--	Free	Free
11	Telephone card	--	500	500

1.8. Methodology

1.8.1. Software Process Model

The development of the E-Bid Processing System utilizes the **MERN Stack**, a collection of robust JavaScript-based technologies designed to build high-performance, scalable web applications. The following technologies are employed:

React.js (Frontend): Used for building a dynamic and interactive User Interface (UI). It allows for a Single Page Application (SPA) experience where content updates instantly without refreshing the page, utilizing a component-based architecture.

Node.js (Runtime Environment): An open-source, cross-platform JavaScript runtime environment that executes code outside the browser. It allows the system to handle multiple concurrent connections efficiently using an event-driven, non-blocking I/O model.

Express.js (Backend Framework): A minimal and flexible web application framework for Node.js. It is used to build the **RESTful APIs** that manage the business logic and facilitate communication between the React frontend and the MongoDB database.

- ✧ **MongoDB (Database):** A NoSQL, document-oriented database used to store procurement data in JSON-like documents. It was chosen for its flexibility in handling complex bid documents and its high scalability compared to traditional relational databases.
- ✧ **Tailwind CSS / Bootstrap:** Utilized for styling the user interface to ensure a professional and modern look. These frameworks ensure the system is **Responsive**, meaning it functions perfectly across desktops, tablets, and smartphones.
- ✧ **JavaScript (ES6+):** The primary programming language used across the entire stack (both client-side and server-side), ensuring a unified and consistent codebase.
- ✧ **Vite:** The modern build tool used to bundle the React frontend, providing a fast development environment and optimized production builds.
- ✧ **Axios:** A promise-based HTTP client used to connect the React frontend with the Express backend APIs for data fetching and submission.

1.8.2. Tools and Techniques

- **MS Office Visio**: Used for creating professional architectural diagrams, flowcharts, and detailed system designs because of its specialized toolsets for software modeling.
- **Lucidchart / StarUML**: Used as additional cross-platform modeling tools for generating complex UML diagrams, including Use Case, Sequence, and Class diagrams, to visualize the system's object-oriented structure.
- **Draw.io (diagrams.net)**: An open-source alternative to Visio used for designing system architectures and activity diagrams, ensuring collaborative access during the design phase.
- **MS Office Word**: Used for the comprehensive documentation of the project report, gathering all requirements, analysis, and implementation details.
- **MS Office PowerPoint**: Used for preparing the visual presentations of the project for review and defense sessions.

1.9. Roles and Responsibilities

Table 2:roles and responsibilities

Student Name	Assigned Role	Key Responsibilities
AMARECH GUADIE	**Project Manager & Frontend Lead**	Oversight of project timelines, designing UI mockups in Figma, and developing the React.js frontend components.
SHISHIG DESALEGN	**Backend Developer**	Developing the server-side logic using Node.js and Express.js, and creating RESTful API endpoints.
G/MARIAM ABIRHAM	**Database Administrator**	Designing MongoDB document schemas, managing Mongoose models, and ensuring data integrity.
MESERET ANDARGIE	**Full-Stack Integrator**	Connecting the React frontend with the Express backend and implementing JWT authentication.

Student Name	Assigned Role	Key Responsibilities
AKLILU EBAKUM	**Quality Assurance & Security**	Performing system-wide testing (Unit/Integration), securing bid data, and documenting the final report.

1.10. Work breakdown Structure

The project development is organized into the following major phases and tasks:

Phase 1: Project Initiation & Requirement Analysis

- ✧ **System Investigation**: Studying the current manual bid processing system used at DTU IOT to identify bottlenecks.
- ✧ **Requirement Gathering**: Conducting interviews and observations with stakeholders (purchasing manager, storekeepers, and finance) to define system needs.
- ✧ **Feasibility Study**: Conducting technical, operational, economic, and schedule feasibility assessments.
- ✧ **SRS Documentation**: Preparing the Software Requirement Specification (SRS) including functional and non-functional requirements.

Phase 2: System Analysis & Design

- ✧ **UML & Architecture Modeling:** Constructing analysis-level diagrams including Use Case, Sequence, and Activity diagrams. In addition to standard UML, define a **REST API Architecture diagram** to map the communication between the React frontend and Node.js/Express backend.
- ✧ **NoSQL Database Design (Data Modeling):** Designing a flexible, document-oriented schema using **MongoDB and Mongoose**. This involves defining JSON-like schemas for suppliers, requisition forms, and bid records, focusing on data embedding and referencing strategies for optimal performance.
- ✧ **Component-Driven Interface Prototype:** Designing a user-friendly, responsive **Single Page Application (SPA)** interface using **React.js**. This includes creating

high-fidelity prototypes using **Tailwind CSS** or **Material UI** for modern styling and defining a component hierarchy for modularity.

- **Security & Authentication Planning:** Designing robust security protocols including **JSON Web Tokens (JWT)** for secure user sessions, password hashing using **Bcrypt**, and implementing field-level encryption for sensitive financial offers within the MongoDB collections.

Phase 3: System Implementation (Development)

- **Frontend Development (React.js):** Developing a dynamic and interactive Single Page Application (SPA) using **React.js**. This involves building reusable UI components, managing application state with **Hooks (useState, useEffect)** or **Redux**, and ensuring a responsive design using **Tailwind CSS** or **Bootstrap**.
- **Backend Development (Node.js & Express.js):** Implementing the server-side logic and business rules using **Node.js** and the **Express.js** framework. This includes building **RESTful APIs** to handle bid submissions, user authentication via **JWT (JSON Web Tokens)**, and procurement workflows.
- **Database Integration (MongoDB & Mongoose):** Establishing a connection to **MongoDB** (a NoSQL database) and using the **Mongoose ODM** (Object Data Modeling) to define data schemas. This ensures efficient storage and retrieval of unstructured bid documents, supplier profiles, and requisition forms.
- **Environment & Runtime Configuration:** Configuring the **Node.js runtime environment** and managing dependencies using **NPM** (Node Package Manager).

Phase 4: Testing & Quality Assurance

- **Unit & Integration Testing**: Testing individual system functions (e.g., login, registration, and file upload) to ensure they meet requirements.
- **Security Testing**: Verifying authentication and authorization processes to prevent unauthorized access.
- **Error Handling**: Identifying and resolving any bugs or logical inconsistencies in the bid processing workflow.

Phase 5: Deployment & Documentation

- **System Deployment**: Final installation of the web-based system for active use at DTU IOT.
- **Project Documentation**: Finalizing the project report using MS Word and preparing the defense presentation using MS PowerPoint.
- **User Training**: (Optional) Providing necessary guidance for stakeholders to navigate the new automated platform.

1.11. Schedule

Table 3 time schedule

Task ID	Phase / Task Description	Estimated Duration	Dependency
P1	**Requirement Analysis & Investigation**	**3 Weeks**	**None**
T1.1	Investigation of current manual system at DTU IOT	1 Week	--
T1.2	Stakeholder interviews (Purchasing manager, storekeepers)	1 Week	T1.1
T1.3	Feasibility study and SRS documentation	1 Week	T1.2
P2	**System Analysis & Design**	**4 Weeks**	**Phase 1**
T2.1	UML modeling (Use Case, Sequence, Component diagrams)	2 Weeks	P1
T2.2	**NoSQL Schema Design (MongoDB & Mongoose)**	1 Week	T2.1
T2.3	**UI/UX Design & Component Hierarchy (React Wireframes)**	1 Week	T2.1
P3	**System Implementation (Development)**	**6 Weeks**	**Phase 2**

Task ID	Phase / Task Description	Estimated Duration	Dependency
T3.1	**Frontend Development (React.js & State Management)**	2 Weeks	P2
T3.2	**Backend Development (Node.js, Express & REST APIs)**	3 Weeks	T3.1
T3.3	**Database Integration & JWT Security Implementation**	1 Week	T3.2
P4	**System Testing & Debugging**	**2 Weeks**	**Phase 3**
T4.1	Unit and API integration testing (using Postman/Jest)	1 Week	P3
T4.2	User acceptance testing and final debugging	1 Week	T4.1
P5	**Deployment & Final Reporting**	**2 Weeks**	**Phase 4**
T5.1	**System deployment (Node.js Environment/Cloud Hosting)**	1 Week	P4
T5.2	Final documentation and presentation (Word, PPT)	1 Week	T5.1

2. Chapter 2: Study of Existing System

2.1. Introduction

This chapter is primarily concerned with the system analysis phase, providing a detailed description of the existing manual system and the newly proposed web-based system. It explores the business rules followed by the current process and identifies its core strengths and weaknesses. Furthermore, this section briefly outlines the functional requirements of the existing workflow and provides a high-level description of the proposed system's functionality

2.2. Existing System

The current bidding process at **Debre Tabor University** begins with a purchase requisition. When a department requires an item, a request is initiated and sent for approval to the scientific director. The workflow involves several manual checks:

- ✧ **Storekeeper Check**: Verifies if the requested item is already available in the inventory.
- ✧ **Director Approval**: If the item is not available and deemed necessary, the director orders the purchasing team to proceed.
- ✧ **Tender Preparation**: The purchasing team manually prepares tender forms and announces them to potential suppliers.
- ✧ **Market Analysis**: The team also manually gathers current market prices for each requested item.

Currently, this system is entirely non-automated, which makes the procurement process difficult, time-consuming, and prone to delays.

2.3. Organization Structure

2.3.1. Players of the Existing System

In the procurement workflow, various "players" or actors interact to complete the bidding and purchasing process. Each role has specific responsibilities:

- **Users (Departments)**
  - ✓ Initiate the process by requesting goods to purchase.
  - ✓ Define the demand for specific items needed for university purposes.
- **Scientific Director**
  - ✓ Approve requisitions initiated by departments.
  - ✓ Provide and reply to feedback regarding procurement requests.
  - ✓ Direct the purchasing team to proceed if items are not in stock and are necessary.
- **Purchasing Team**
  - ✓ Collect and organize purchasing items and validate them.
  - ✓ Manage the tender summary, open tenders, and evaluate rankings.
  - ✓ Select the final winner of the auction.
- **Storekeeper**
  - ✓ Check the availability of requested items in the current stock.
  - ✓ Manage inventory, control stock levels, and receive new goods.
- **Finance Department**
  - ✓ Check the available budget for the requested items.
  - ✓ Manage messages and financial communication within the system.
- **Suppliers**
  - ✓ View tender notices and participating in the bidding process.
  - ✓ View selection results and receive payments upon winning.

2.3.2. Organizational Workflow

The organization of these roles follows a hierarchical structure where the process flows from the top management down to the external vendors:

- ✧ **Management Level**: Scientific Director (Final approval authority).
- ✧ **Administrative Level**: Finance and Storekeeper (Validation of resources and budget).
- ✧ **Operational Level**: Purchasing Team and Purchasing Officer (Execution of the bid process).
- ✧ **External Level**: Suppliers (Participants in the competitive bidding).

This structure ensures that every purchase is verified for need (by the Director), availability (by the Storekeeper), and funding (by Finance) before the Purchasing Team initiates a tender

2.4. Services provided

2.4.1. Service Provided By The Existing System

The current manual system at DTU IOT provides basic procurement services, though they are restricted by physical and administrative limitations:

- ✧ **Manual Requisition**: Departments initiate service by physically filling out paper-based purchasing requisition forms.
- ✧ **Inventory Verification**: The storekeeper manually checks physical records to verify the availability of requested items.
- ✧ **Budget Validation**: The finance department provides a service of checking and confirming available funds for a specific purchase.
- ✧ **Physical Tender Announcements**: The purchasing team prepares and posts tender notices on physical notice boards for local suppliers.
- ✧ **On-Site Bid Submission**: Suppliers must physically come to the campus to register and submit their bid documents.

2.4.2. Service Provided By The Proposed E-Bid System

The proposed web-based system automates the core procurement cycle to improve efficiency and reach. The specific services include:

Core functionalities

- **Online Supplier Registration**: Allows suppliers to register remotely, creating a digital database of potential vendors.
- **Electronic Tender Management**: Provides the service of uploading tender documents digitally and setting automated deadlines for submissions.
- **Digital Requisition Processing**: Departments can submit purchase requests online, which are then routed automatically for approval.
- **Automated Evaluation Support**: The system provides the service of evaluating the financial portion of bids automatically to ensure fairness.
- **Winner Notification**: Automatically notifies the winning bidder and informs unsuccessful participants of the results.

Support and Security Services

- **Account Management**: Administers different access roles for users (Scientific Director, Finance, Storekeeper, etc.) to ensure duty segregation.
- **Data Encryption**: Provides a security service that encrypts sensitive supplier information to prevent unauthorized modifications or "corruption".
- **Real-Time Communication**: Enables internal users to send and receive messages regarding procurement status within the platform.
- **Report Generation**: Generates various procurement and auction summary reports as demanded by university management.

2.5. Users

The system is designed with a multi-user architecture where each user has specific privileges to ensure a secure and organized procurement workflow:

1. **System Administrator**
   - Manages user accounts (creates, updates, and deactivates accounts).

- ✧ Assigns roles and permissions to university staff and suppliers.
- ✧ Maintains the overall system security and database integrity.

2. **Scientific Director (Approver)**

- ✧ Reviews and approves or rejects purchase requisitions sent by departments.
- ✧ Monitors the status of major procurement activities within the Institute.

3. **Purchasing Team / Officer**

- ✧ The primary operators of the system.
- ✧ Uploads tender documents and bid announcements.
- ✧ Evaluates supplier documents and identifies the winning bidder.
- ✧ Generates auction summary reports.

4. **Storekeeper**

- ✧ Verifies if requested items are available in the physical store before a purchase is authorized.
- ✧ Updates the inventory records once new items are received from a winning supplier.

5. **Finance Department**

- ✧ Verifies the availability of budget for requested items.
- ✧ Handles the financial communication and documentation related to the bid.

6. **Department Heads (Requisitioners)**

- ✧ Staff members from various departments (e.g., Mechanical, Electrical, IT) who initiate purchase requests for equipment or materials.

7. **Suppliers (Bidders)**

- ✧ External users who register on the platform.

- ✧ Browse active tenders and download bid documents.
- ✧ Submit their financial and technical proposals electronically.
- ✧ View the results of the bid evaluation.

2.6. Business rule identification

A business rule is a statement that describes a business policy and procedure. They cannot break down further this rules. It describes the sequence of operations associated with data in the database to carry out the rule.

1. All users who want to get service of the system they must fill the request form.
2. The form that is filled by the user must be checked by an administrator and returned back to the user.
3. The purchasing team must post the auction announcement for the supplier before the purchasing.
4. The suppliers who need to compete in the bidding process must be registered and the information must be accessible by the supplier.
5. Any supplier who able to supply is computed in the auction process without any restriction.
6. The suppliers who supply the material for the campus first they must win the auction.
7. The finance must be paid money for the suppliers to get the material.
8. The administrator must manage the whole process of the procurement system.
9. The administrator must close the auction process after the end of this activity and the procurement system end.

2.7. Existing infrastructure

The infrastructure at Debre Tabor University serves as the backbone for the proposed E-Bid Processing system. While the current procurement process is manual, the university possesses the necessary hardware and network capabilities to host a web-based platform.

A. Network Infrastructure

- **Local Area Network (LAN)**: The university is equipped with a functional LAN connecting various departments, including the Purchasing Team, Finance, and the Scientific Director's office.

- **Internet Connectivity**: High-speed broadband internet is available throughout the IOT campus, allowing the system to be accessible to external suppliers for remote bidding.

- **WiFi Coverage**: Wireless access points are distributed across administrative buildings, facilitating mobile access for staff members.

B. Hardware Infrastructure

- **Server Infrastructure:** Debre Tabor University (DTU) maintains a centralized server environment capable of running the **Node.js runtime engine** and hosting **MongoDB instances**. Unlike traditional setups, the infrastructure supports a non-blocking, event-driven architecture, ensuring high concurrency for multiple simultaneous bidders. The server environment is configured to handle the **Express.js backend** and serves the **React.js production build** via high-speed data delivery.
- **Workstations (Client-Side):** Most departments are equipped with modern desktop computers and laptops that meet the memory requirements for running modern web browsers (e.g., Chrome, Firefox, or Edge). These workstations act as the execution environment for the **React-based Single Page Application (SPA)**, ensuring that the heavy lifting of UI rendering is handled by the client's hardware for a faster user experience.
- **Power & Network Reliability:** The university utilizes backup generators and Uninterruptible Power Supplies (UPS) to ensure the **Node.js server** remains online

24/7. This is critical for the e-bid system, as the automated countdown timers for bid closings (managed by the server) require constant uptime to ensure fairness and data integrity for all suppliers.

C. Physical Facilities

- **ICT Center**: A dedicated ICT support team is available to handle hardware maintenance and network troubleshooting.
- **Procurement Office**: A physical office space exists where the purchasing team currently operates, which will serve as the primary hub for managing the new digital workflow.

2.8. Proposed System

The proposed **E-Bid Processing System** is a modern, web-based automated platform designed to replace the current paper-based procurement workflow at Debre Tabor University. Built using the **MERN stack (MongoDB, Express.js, React, and Node.js)**, the system digitizes the entire lifecycle of a tender—from the initial department requisition to the final award of a contract. By leveraging a **document-oriented MongoDB database** and a **Node.js/Express RESTful API**, the platform provides a highly scalable and real-time environment that ensures data integrity and seamless interaction for all procurement stakeholders.

Core Objectives of the Proposed System

- **Automation**: Eliminates manual paperwork, reducing manpower and material wastage.
- **Transparency**: Provides a clear digital trail of approvals and bid evaluations.
- **Security**: Implements encryption to protect sensitive supplier financial offers until the official bid opening time.
- **Accessibility**: Allows suppliers to participate in tenders from anywhere, increasing competition and lowering costs for the university.

Key Features and Functionality

- **Electronic Requisition**: Department heads can submit purchase requests online, which are instantly visible to the Storekeeper and Scientific Director for validation.
- **Digital Tender Announcement**: The Purchasing Team can post tender notices and upload technical specifications directly to the web portal.
- **Supplier Portal**: Registered suppliers can log in, download tender documents, and submit their financial and technical bids through a secure interface.
- **Automated Evaluation Support**: The system assists in ranking bidders based on predefined criteria, reducing human error in the selection process.
- **Status Tracking**: Users can track the progress of their requests in real-time
- **Report Generation**: The system can automatically generate auction summaries and procurement history reports for university management.

3. Chapter 3: Software Requirement Specification

3.1. Introduction

This chapter is the main and soul of our proposed system. We focus mainly on the system features and analysis model. It deals with the proposed system and finds out the functional and non-functional requirements of the automated system; also try to solve the existing system problem by analyzing each and every detail of the system.

3.2. General constraints

The development and implementation of the E-Bid Processing system are governed by several constraints categorized into technical, environmental, and administrative factors.

1. Technical Constraints

- **Browser Compatibility**: Since the system is web-based, it must be compatible with standard modern browsers (Google Chrome, Mozilla Firefox, Microsoft Edge). It may not perform optimally on older versions of Internet Explorer.

- ✧ **Language Constraint**: The system interface and all generated reports will be in **English**, as it is the primary language for technical and academic documentation at DTU.
- ✧ **Connectivity**: The system requires a stable internet or intranet connection. Users in areas with zero connectivity will not be able to access the bidding platform or submit requisitions.

2. Hardware & Software Constraints

- ✧ **Runtime & Database Dependency:** The system's availability is strictly dependent on the **Node.js runtime environment** and the **MongoDB database instance** being operational. Unlike traditional setups, the system utilizes a non-blocking architecture; however, seamless access requires the backend **Express.js API** to be active. Any interruption in the Node.js process or the NoSQL database cluster will render the application inaccessible.
- ✧ **Storage & Scalability Limits:** The capacity for storing large technical and financial bid documents (primarily in PDF format) is governed by the **MongoDB GridFS** configuration or the physical storage assigned to the server. While MongoDB is highly scalable, the total volume of data remains limited by the university's storage infrastructure or the cloud-based storage tier utilized for managing binary large objects (BLOBs).

3. Security Constraints

- ✧ **Authentication**: No user can access the system's core features without a verified username and password.
- ✧ **Encryption**: Sensitive financial data must remain encrypted until the specific "Bid Opening" date and time. Any premature access would violate procurement laws.

4.Operational & Policy Constraints

- **Legal Compliance**: The system must adhere to the **Ethiopian Federal Government Procurement and Property Administration** proclamations and university-specific bylaws.
- **User Training**: The effectiveness of the system is constrained by the digital literacy of the users. Staff and suppliers must have basic computer skills to navigate the PHP-driven interface.
- **Power Supply**: Despite the infrastructure having UPS systems, prolonged power outages in the region remain a significant operational constraint.

3.3. Specific Requirements

These requirements provide a detailed description of the external interfaces and the environment in which the system operates.

3.3.1. External Interface Requirements

The external interface requirements identify the interactions between the E-Bid system and the entities (users, hardware, and software) that exist outside the system's boundary.

3.3.1.1. User Interfaces

The system shall provide a dynamic, component-based **Single Page Application (SPA)** interface using **React.js**. This ensures a fluid user experience without page reloads. Key features include:

- **Responsive Component Architecture:** Built using **React-Bootstrap** or **Tailwind CSS**, the UI will utilize a grid-based system to ensure the bidding portal is fully accessible across Desktops, Tablets, and Smartphones.
- **Dynamic Navigation & Routing:** Implementing **React Router** to provide a persistent sidebar or top-bar menu. This allows seamless transitions between "Live Bids," "Department Requisitions," and "Procurement Analytics" while maintaining application state.

- **State-Driven Forms & Validation:** Utilizing **React Hook Form** or **Formik** integrated with **Yup** for schema-based validation. The system will provide instant, client-side feedback (e.g., for technical specification uploads or financial quote formatting) before the data is even sent to the Node.js backend.
- **Reusable UI Components:** Implementation of a consistent theme through reusable React components (Buttons, Modals, Tables). This ensures that university branding and professional administrative layouts are uniform across all procurement modules.

3.3.1.2. Hardware Interfaces

The system does not require specialized hardware but must interact efficiently with:

- **Computers/Laptops**: Standard PCs used by university staff and suppliers to access the web portal.
- **Printers**: Support for generating and printing procurement summaries, award letters, and bid results in PDF format.
- **Storage Media**: Interaction with server-side hard drives for the permanent storage of uploaded tender documents and supplier licenses.

3.3.1.3. Software Interfaces

The E-Bid system is built on a modern JavaScript runtime environment and must interface with the following software components:

- **Runtime & Operating Systems:** The server-side logic interfaces with the **Node.js runtime engine**, which can be deployed on Linux (e.g., Ubuntu/CentOS) or Windows Server environments. The client-side interface is entirely OS-independent, delivered as a **React.js application** accessible through any modern web browser (Chrome, Firefox, Safari, Edge).
- **Database Interface (ODM):** Instead of direct SQL queries, the system interfaces with **MongoDB** via **Mongoose (Object Data Modeling)**. This allows for seamless data retrieval and storage using JavaScript objects, providing high performance for complex procurement data and flexible document structures.
- **Application Server Framework:** The system interfaces with the **Express.js framework** to manage the request-response lifecycle. This replaces the traditional

Apache/PHP setup, using a non-blocking I/O model to handle simultaneous API calls from multiple bidders and university departments.

- **File Processing & PDF Rendering:** The system will interface with **Multer** (for handling multi-part form data/file uploads) and browser-based PDF rendering engines. It will also utilize PDF generation libraries (such as **PDFKit** or **jsPDF**) on the server-side to dynamically create procurement reports and tender award certificates.

3.3.1.4. Communications Interfaces

This section defines the protocols and network requirements:

- **TCP/IP Protocol**: The standard communication protocol for data exchange over the university network and the internet.
- **HTTP/HTTPS**: The system will use Hypertext Transfer Protocol Secure (HTTPS) to ensure that all data transmitted between the supplier's browser and the DTU server is encrypted.
- **Email Integration (SMTP)**: Interface with Simple Mail Transfer Protocol to send automated notifications to winners and alert department heads of requisition statuses.
- **LAN/WAN**: The system will be hosted on the university's Local Area Network but will be accessible via the Wide Area Network for external suppliers.

3.4. Functional Requirements

Functional requirements are a mandatory requirement that must be incorporated into our system.

It is a description of activities and services that the system must provide; is clear that the new system has to perform all the tasks done by the current system without changing the rules and in a cost-effective way. The following are the functional requirements that must be fulfilled by our proposed system.

- **Supplier Registration*:*** the supplier must be registered to participate in the bid process. The system handles the supplier's information in a secure way by encrypting their data to deny further modification.

- ✓ The system must handle a database of registered suppliers.
- ✓ It should handle the required details about each supplier.

- ✧ **Upload tender file*:*** the purchasing team must upload a tender file for the supplier by considering the criteria. In order to upload the tender file, the criteria are prerequisite and must be prepared.
- ✧ **Disable tender file*:*** the bid process has a start and end date; so that after the end date of the process the purchasing team must be disable the tender file from other user. This helps the sytem by avoiding suppliers that are registered after the end date of auction.
- ✧ **Arrange requisition*:*** users can give suggestions about demanded materials to the purchasing team. Then the purchasing team arranges the users demand by considering the available data in the store and the available budget for that purpose.
- ✧ **Approve Requisition*:*** after arranging the user request the purchasing team update the request and save into the database. Then scientific director approves the requisition.
- ✧ **Manage Account*:*** managing an account is the system administrators' activity. He/she can add users to the system, update their information when it is needed and remove users from the system if they have no longer needed to the system. The system administrator changes the user's password if it is the user's need.
- ✧ **Delete*:*** the purchasing team can be delete information that are not needed any more for their bid process.
- ✧ **Send a message:** an actor can send a message to the purchasing team, scientific director, finance, and administrator and also send a general suggestion to the administrator.
- ✧ **View registered supplier information*:*** the purchasing team sees all the recent and needed as well as registered suppliers for the evaluation by using a decryption key.

- ✧ **Announce the winner*:*** the purchasing team announces the winner of the auction after the evaluation of the bid document. The supplier knows who the winner is and who the fallers are.

- ✧ **Evaluate bidders*:*** the evaluation of bidders have two parts technical (30%) and financial (70%) evaluation. But in our case the system evaluates only the financial part. The technical part evaluation is done by the purchasing team.

- ✧ **Input Requirements:** when we say input requirement of the system, we mean that the process or preconditions that should be fulfilled to produce the desired output: that is the input starting from users requisition up to get items. These input requirements are:

    - ✓ Requisition
    - ✓ Registration
    - ✓ Evaluation
    - ✓ Register available materials.

- ✧ **Data storage and Retrieval:** the system store supplier record and information related to the auction process and should retrieve the information when necessary. The retrieval of information should be easy and data should be saved properly in a well-organized database server so that the process of data retrieval would be simple and fast.

3.5. Use Case Design

A system use case describes the interaction between the user and the system in a more detailed way than an essential use case. Although the System Use Case provides a more detailed description of the steps that the system will perform to fulfill the need of the user.

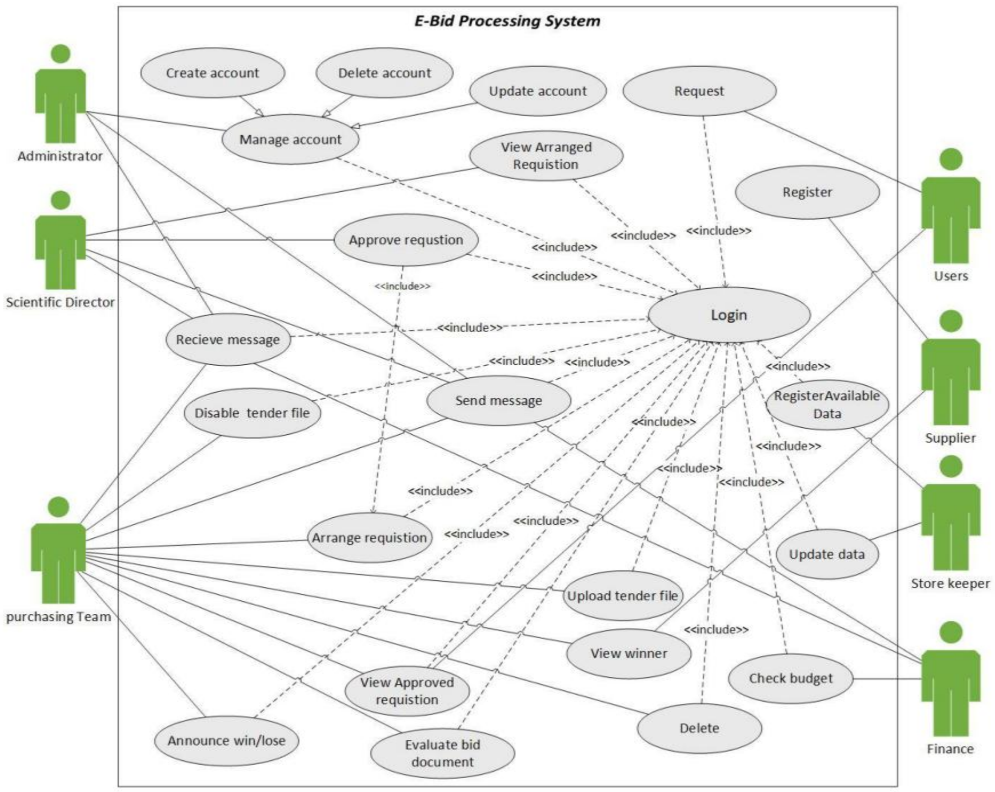


Figure 1 usecase diagram

3.5.1. Actor Identification

Actor identification involves defining the different entities (human or external systems) that interact with the E-Bid Processing System. An actor represents a **role** played by a user.

- ✧ **Key Actors**: Include the **System Administrator** (manages accounts), **Scientific Director** (approves requests), **Purchasing Team** (manages tenders), **Storekeeper** (checks inventory), **Finance** (checks budget), and **Suppliers** (submit bids).
- ✧ **Purpose**: To define the boundaries of the system and ensure every user has a specific set of permissions.

3.5.2. Use case Identification

Use case identification is the process of listing all the high-level functionalities or goals that the actors achieve using the system. It describes **what** the system does without explaining **how** it does it.

- ✧ **Use Cases**: *Register Supplier, Create Tender, Submit Requisition, Evaluate Bid, Generate Award Letter,* and *Post Result.*
- ✧ **Purpose**: To capture the functional requirements in a way that shows the value provided to each actor.

3.5.3. Use case Description

Table 4 :login description

Use case name	Login
Use case ID	SUC#001
Actor	Administrator, Scientific director, purchasing team, storekeeper, user & Finance.
Precondition	The scientific director, User, purchasing team and finance must enter the username and password.
Post Condition	Scientific director, purchasing team, purchasing officer, storekeeper, user, and Finance can access the tasks.
Description	The authentication for authorized user in the system to interact with Bid system.
Trigger	User's wants to login to the system.
Basic course of action	1. The scientific director, purchasing team, storekeeper, user, and Finance want to login to the system 2. They enter username and password 3. They click the login button 4. The system validates the entered information 5. Users' enter into their account. 6. use case End
An alternative course of action	A: They cannot access the system A4: The system informed them an error message A6: use case end

Table 5 create Account

Use case name	Create account
Use case ID	SUC#002
Actor	Administrator
Precondition	The administrator must create the account.
Postcondition	The actor who gets the account continue their tasks
Description	This activity is performed when the administrator wants to create a new account for users.
Trigger	The administrator wants to create user accounts.
Basic course of action	1. The administrator wants to create the account 2. The administrator login to the system. 3. The administrator enters the username and password 4. The administrator clicks create a button 5. The system validates the entered information 6. The system display and save the created account 7. The system sends the success message to the administrator 8. End-use case
An alternative course of action	A: The account cannot be created A5: The system tells the entered information is not valid A8: End-use case

Table 6 update Account

Use case name	update account
Use case ID	SUC#003
Actor	Administrator
Precondition	Account must be created
Postcondition	The system has a secret
Description	This activity is done when the Administrator wants to update accounts for users.
Trigger	The administrator wants to update the user's account.
Basic course of action	1. The administrator wants to update the account 2. The administrator enters the old user name and password. 3. The system verify username and password 4. The administrator enters a new username and password 5. The system validates the entered information and save. 6. The system sends a success message 7. End use-case
An alternative course of action	A: the user do not enter valid username and password A5: the system orders the user in order to enter a valid username and password A7: end use case

Table 7 Request

Use case name	Request
Use case ID	SUC#004
Actor	User
Precondition	The user must be login into the system
Postcondition	The scientific director approves the user requisition
Description	This activity is performed when users want/demand for goods or if users or departments want to buy materials for a purpose.
Trigger	Users want to buy materials.
Basic course of action	1. The user wishes to ask request 2. The user enters the username and password 3. The system verifies the user's username and password 4. The user fills the requisition form 5. The system verifies the filled requisition forms 6. The system saves the requests to the database 7. The system sends the success message to the user 8. Use case end
An alternative course of action	A: The user do not have access to privilege A3: The system verifies the user does not have access privilege A8: use case end B: the user could not fill the requisition forms correctly B5: The system verifies the user filled the requisition forms correctly B8: use case end

Table 8:Arrange requistion

Use case name	Arrange requisition
Use case ID	SUC#005
Actor	Purchasing team
Precondition	The requisition must be requested by the user and the budget must be known.
Post condition	The scientific director approve a requisition
Description	The purchasing team should arrange the requests base on the information that they get from storekeeper and finance staffs. If the material is available in stoke they reject requisition but if the available material is not enough and there is sufficient budget they will continue the process.
Trigger	Purchasing team wants to arrange requests.
Basic course of action	1. The purchasing team want to arrange the requisition 2. Purchasing team see the requisition 3. Then purchasing team arrange the requisition 4. If the request not available in stoke and if there is sufficient budget purchasing The team accepts the request. 5. If the request is available but not enough the purchasing team checks the Quantity. 6. The system calculates the quantity 7. The system updates request. 8. End use-case
An alternative course of action	A: If the request available A4: if available material enough the purchasing team reject the request A8: End use-case

Table 9 :view Arrange requistion

Use case name	View arranged requisition
Use case ID	SUC#006
Actor	Scientific director
Precondition	The user must request and purchasing team arrange requisition.
Post condition	Scientific director approves requisition.
Description	The scientific director views the request of users that is arranged by purchasing officers in order to approve it.
Trigger	The scientific director wants to views requisition to approve it.
Basic course of action	1. The scientific director wants to view arranged user request. 2. Scientific director enters username and password 3. System verify username and password 4. Scientific director view the requisition 5. Use case end
An alternative course of action	No alternative course of action

Table 10 :Approve requisition

Use case name	Approve requisition
Use case ID	SUC#007
Actor	Scientific director
Precondition	The scientific director must be login into the system
Postcondition	The user knows the result of requisition
Description	The scientific directorate has the ability to approve the requisition to make a fair distribution of materials between all users or departments.
Trigger	The scientific director wants to decide the decision for the requested items.
Basic course of action	1. The scientific director wants to approve the request 2. The scientific director enters the username and password 3. The system verifies the username and password 4. The scientific director see the requisition 5. The scientific director check the filled requisition form 6. The scientific director approve the requisition 7. The scientific director updates the information. 8. The information is stored in a database. 9. Use case end
An alternative course of action	A: incorrect requisition form A8: the system sends reject message. A9: use case end

Table 11:view Approved requistion

Use case name	View approved requisition
Use case ID	SUC#008
Actor	Purchasing team, user
Precondition	The requisition must be approved
Postcondition	Purchasing team uploads tender files.
Description	User and purchasing team views the requisition that is approved by the scientific director in order to know the final accepted request.
Trigger	User and purchasing officer wants to know the final accepted request.
Basic course of action	1. User and purchasing team wants to know the approved request 2. User and purchasing team enters username and password 3. The system verify username and password 4. User and purchasing team views approved the request 5. Use case end
An alternative course of action	No alternative course of action

Table 12:Upload tender file

Use case name	Upload tender file
Use case ID	SUC#009
Actor	Purchasing team
Precondition	The Purchasing team must create the tender file
Postcondition	The supplier see the uploaded tender file
Description	This activity is performed when the purchasing team wants to post the prepared tender form to the public.
Trigger	The Purchasing officer wants to upload tender files.
Basic course of action	1. The Purchasing team want to upload the tender file 2. The Purchasing team verify the tender file 3. Purchasing team set deadline. 4. The Purchasing team upload the tender file 5. The supplier see the uploaded tender file 6. End use-case
An alternative course of action	No alternative course of action

Table 13:Register

Use case name	Register
Use case ID	SUC#010
Actor	Supplier
Precondition	Supplier view the posted tender
Postcondition	Supplier participate in the auction
Description	These are the first formality expected from the suppliers to join our system and have a role in it.
Trigger	A supplier wants to register on the system.
Basic course of action	1. The supplier cite the bid website 2. The supplier click to register menu bar 3. The system prompts to fill the form 4. The supplier fill the form 5. The system validates the entered information 6. Supplier enter the encryption key 7. The system encrypts the information 8. The system registers the supplier 9. The system display success message to supplier 10. Use case end
An alternative course of action	A: The registration form is filled incorrectly A5: The system informed the supplier error message A10: use case end

Table 14:Evaluate Bid document

Use case name	Evaluate Bid Document
Use case ID	SUC#011
Actor	Purchasing team
Precondition	Suppliers must be registered.
Postcondition	Purchasing team announce the winner of the auction
Description	The purchasing team must evaluate the bid documents and they have the ability to accept and reject the suppliers.
Trigger	The purchasing team wants to judge suppliers.
Basic course of action	1. Purchasing team login to the system 2. Purchasing team views registered bidders 3. Purchasing team enter the decryption key 4. The system decrypts information's 5. They view the information filled by the bidders 6. They start the technical evaluation 7. They give evaluation rank 8. After technical evaluation did they start system to continue financial evaluation 9. The system announces the winner of the auction. 10. Use case end
An alternative course of action	A: the suppliers do not have access to privilege. A4: the system error message A10: use case end

Table 15:Announce Winner

Use case name	Announce winner
Use case ID	SUC#012
Actor	Purchasing team
Precondition	Supplier bid document must be evaluated
Postcondition	Supplier deliver the item
Description	This activity is performed after all technical and financial of the supplier is done and the winner is announced to the public by the system.
Trigger	Purchasing team wants to announce the winner.
Basic course of action	1. The system announces the winner 2. End use-case
An alternative course of action	No have alternate course action

Table 16:View Winner

Use case name	View winner
Use case ID	SUC#013
Actor	Supplier
Precondition	Purchasing team must evaluate the bidder's document
Postcondition	The storekeeper collects materials and finance pay for the supplier.
Description	After all the evaluation is done suppliers must see their ranks (the rank will be pass or fail) in order to start the final process (that will be having an agreement with the organization).
Trigger	Suppliers want to know their rank.
Basic course of action	1. The supplier clicks view link. 2. Supplier views the winner. 3. Use case end
An alternative course of action	No alternative course of action

Table 17:Register Available Material

Use case name	RegisterAvailableMaterial
Use case ID	SUC#014
Actor	Storekeeper
Precondition	The material must be checked
Postcondition	Register available materials in the database.
Description	This activity is done when storekeeper wants to store the available materials in the stoke.
Trigger	Storekeeper wants to store available materials.
Basic course of action	1. The storekeeper want to register material 2. The storekeeper see the material or resource 3. The storekeeper register available materials. 4. The system store the data in the database. 5. The system sends a success message to the storekeeper. 6. End use-case
An alternative course of action	A: If the material ID is already available A4: System reject the registration. A6: End use-case

Table 18:Update Data

Use case name	Update data
Use case ID	SUC#015
Actor	Storekeeper
Precondition	Storekeeper must be login to the system
Postcondition	Change the information on available data.
Description	This helps the storekeeper to change the value of available data by incrementing or decrementing the data. This prevents the repetition of data.
Trigger	Storekeeper wants to change the value of data.
Basic course of action	1. The storekeeper login to the system 2. The storekeeper select update menu. 3. The storekeeper fill update form 4. The storekeeper click the update button 5. The system changes the data information 6. The system sends a success message. 7. Use case end.
An alternative course of action	No alternative action.

Table 19:Check Budget

Use case name	Check budget
Use case ID	SUC#016
Actor	Finance
Precondition	The budget must be checked
Postcondition	The system has continued
Description	This activity is done when finance wants to know the available budget, and also if the purchasing team sends a notification to finance to know the budget to start the bid process.
Trigger	Finance wants to know the available budget.
Basic course of action	1. Finance want to check the budget 2. Finance enters username and password 3. The system Finance checks the entered username and password 4. The Finance checks the budget 5. The finance sends detail budget to purchasing team. 6. End use-case
An alternative course of action	A: the accountant do not enter valid username and password A3: the system tells to the accountant in order to enter valid username and password A5: End use-case

Table 20:Send Message

Use case name	Send message
Use case ID	SUC#017
Actor	Administrator, Scientific director, finance and purchasing team
Precondition	The message must be sent
Postcondition	They get the feedback
Description	Each department regarding the bid system can send a message to each other in order to have communication.
Trigger	Departments want to share information.
Basic course of action	1. Administrator, Scientific director, finance and purchasing team want to send a message 2. They select the staff that will accept the message. 3. Write some message 4. The message will be sent 5. The message must be displayed on the perspective of staff. 6. End use-case
An alternative course of action	No alternative course of action

Table 21:Recieve Message

Use case name	Receiving message
Use case ID	SUC#018
Actor	Administrator, Finance, scientific director and purchasing team.
Precondition	The message must be sent
Postcondition	The response the feedback to the received direction
Description	Each department of the system can receive a message that will be sent from other departments of the bid system.
Trigger	Departments want to share information and view that information.
Basic course of action	1. They receive message 2. They view messages 3. Next action will be performed. 4. End use-case
An alternative course of action	No alternative course of action

Table 22: Disable tender file

Use case name	Disable tender file
Use case ID	SUC#019
Actor	Purchasing team
Precondition	The tender file must stay for a limit day
Postcondition	The tender file will be deleted after the allowed date
Description	The purchasing team should disable the tender file after the bid process is ended.
Trigger	Purchasing officer wants to disable the tender file.
Basic course of action	1. The purchasing team want to disable the tender file 2. After the expiration date, the file will be disabled from the supplier. 3. End use-case
An alternative course of action	No alternative course of action

3.6. Sequence Diagram

The sequence diagram is used to depict graphically how objects interact with each other via a message in the execution of use case or operation. They illustrate how the message is sent and received between objects and sequence. It is constructed to visualize and validate different activities between objects by the message.

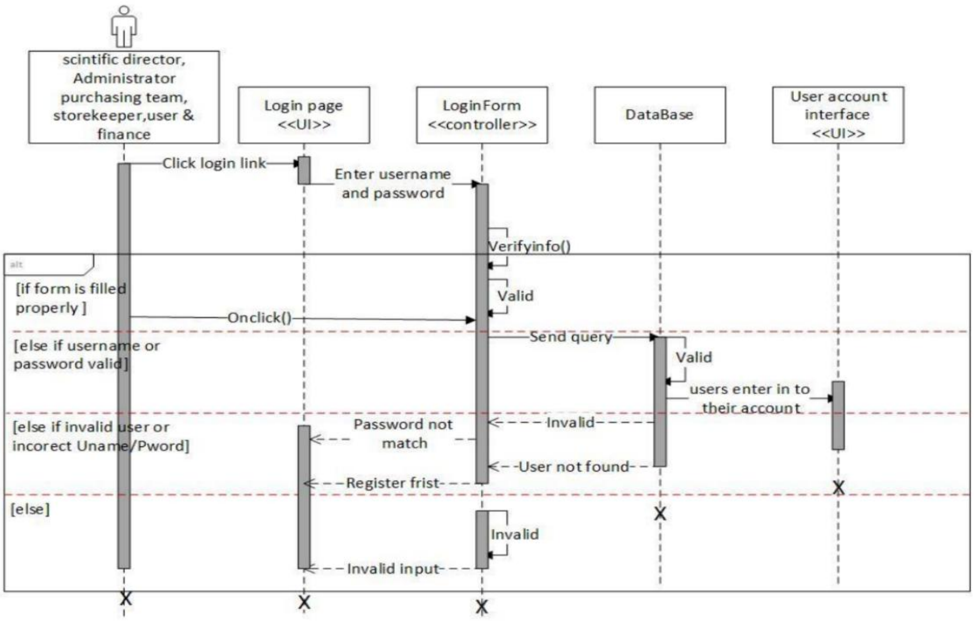


Figure 2:Login sequence diagram

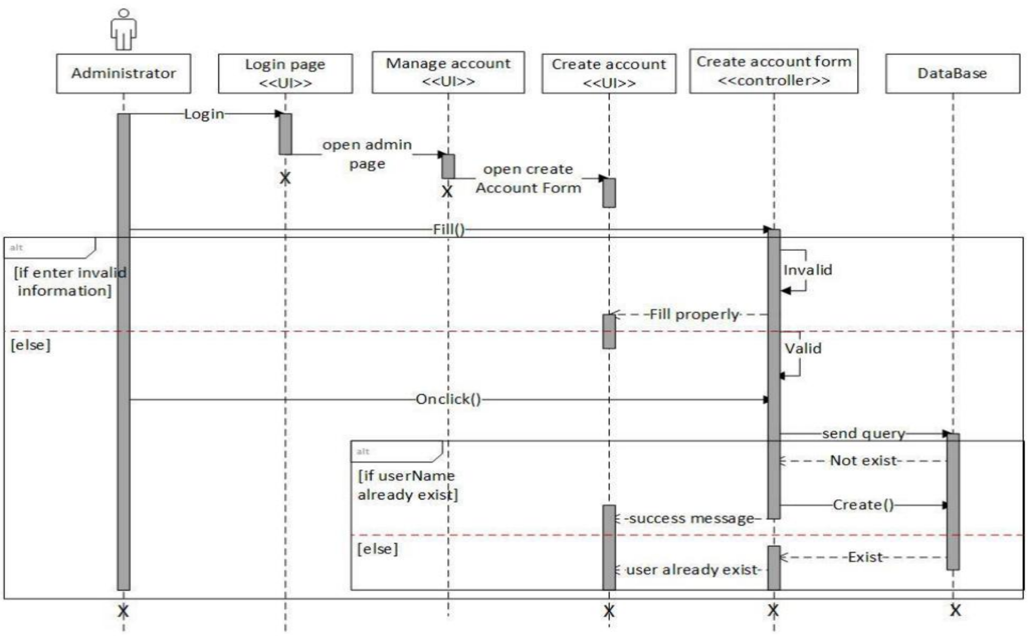


Figure 3:Create account sequence diagram

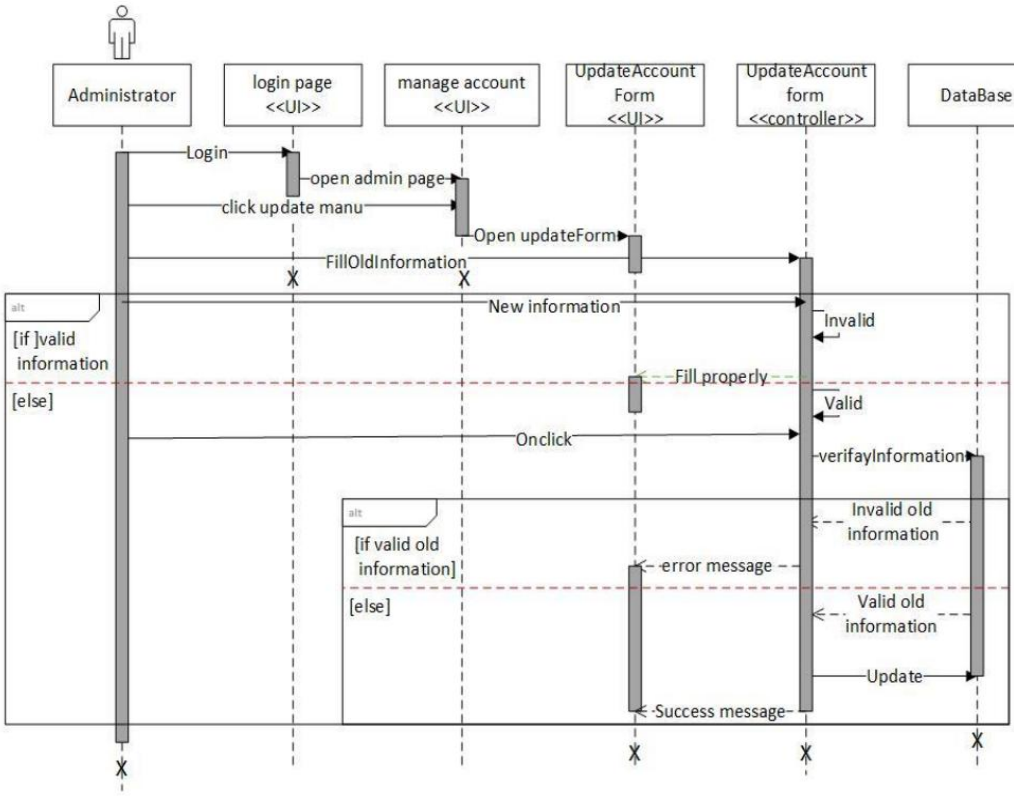


Figure 4:Update account sequence diagram

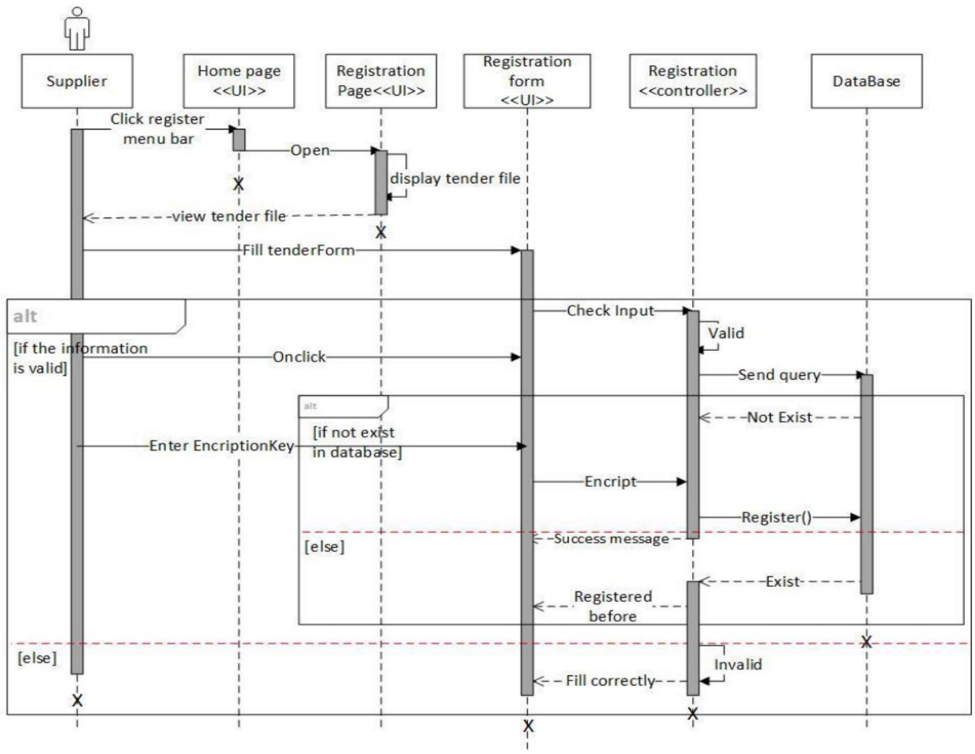


Figure 5:Register sequence diagram

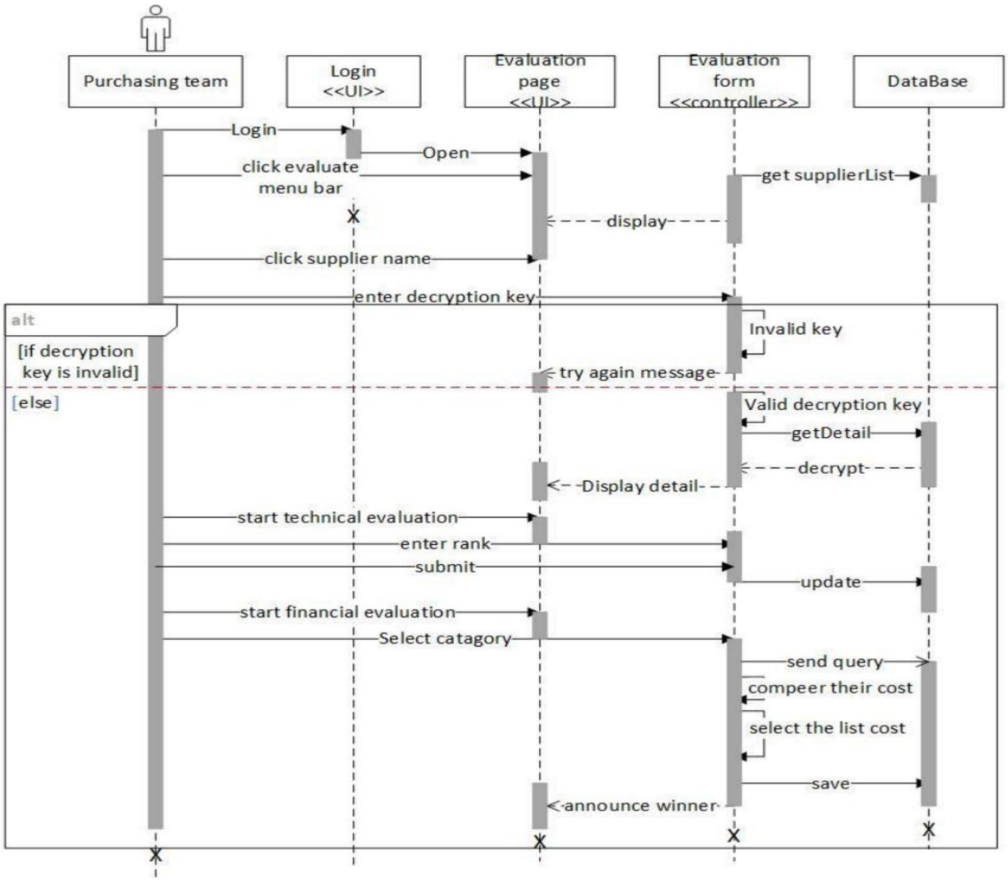


Figure 6:evaluate bidders sequence diagram

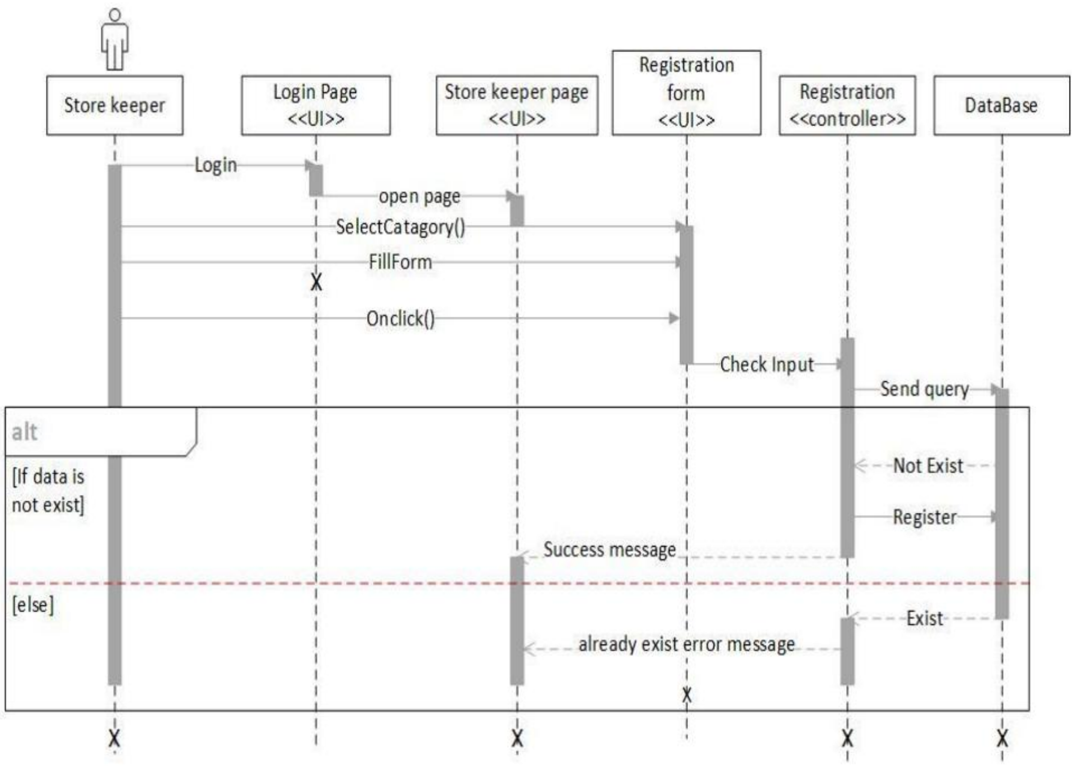


Figure 7:*Register available data sequence diagram*

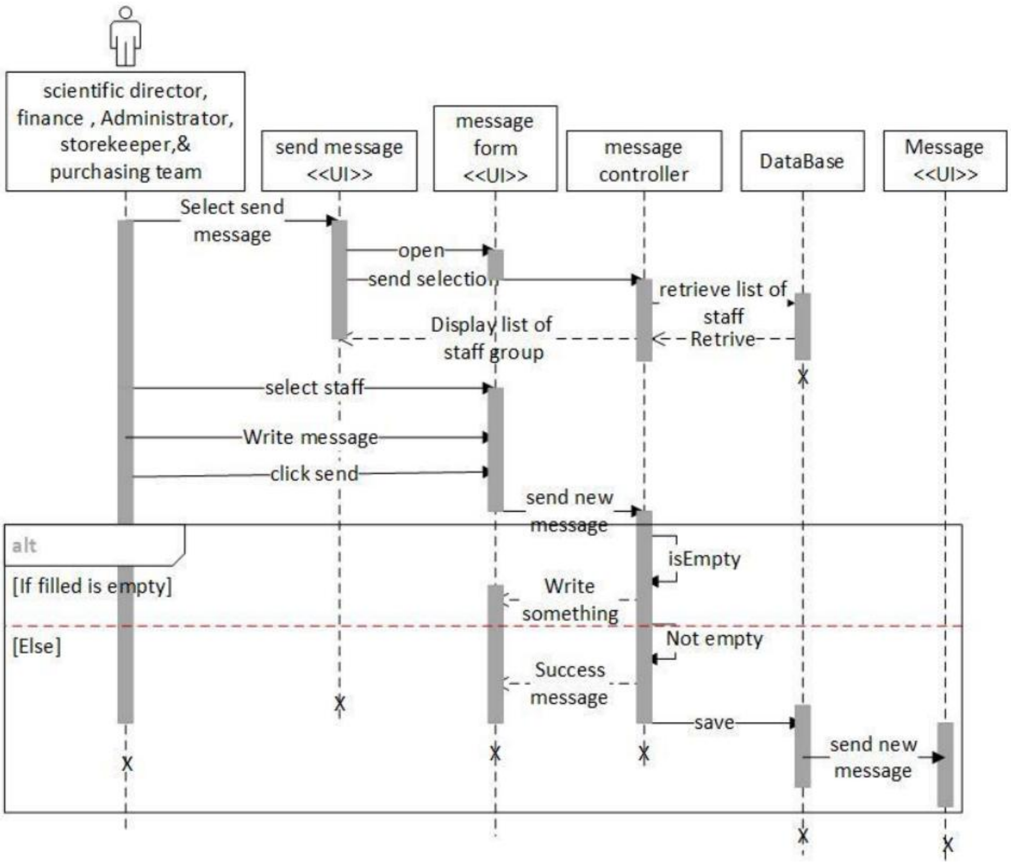


Figure 8:send message sequence diagram

3.7. Activity Diagram

Activity diagrams are used to model the process steps or activities of the system. It illustrates the dynamic nature of the system by modeling the flow of action from activity to activity. An activity represents an operation on some class in the system that results in the change of state in the system. It is a high-level business process, including data flow or to model the logic complex in the system.

The diamond represents a decision point called branch the arrow represent the transition between two activity and the text on the arrow represent a condition that must be fulfilled to process of the transition.

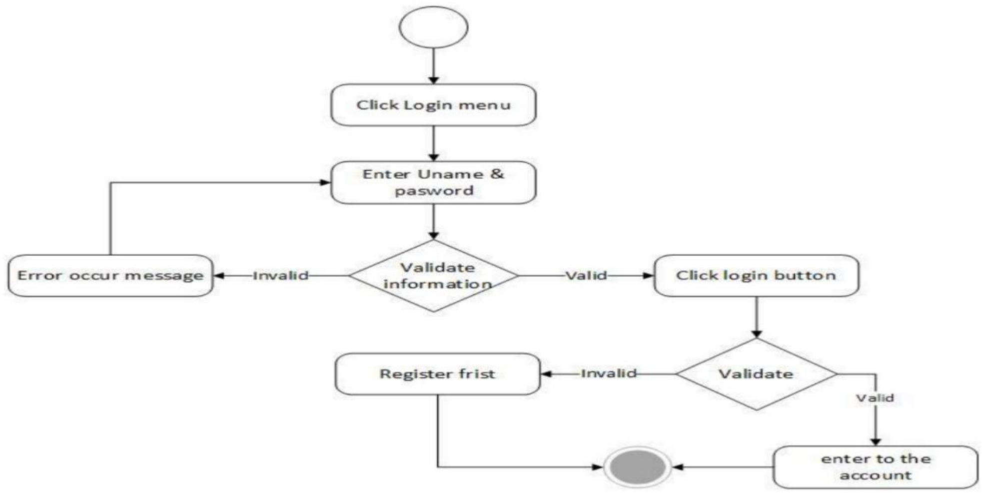


Figure 9:Login activity diagram

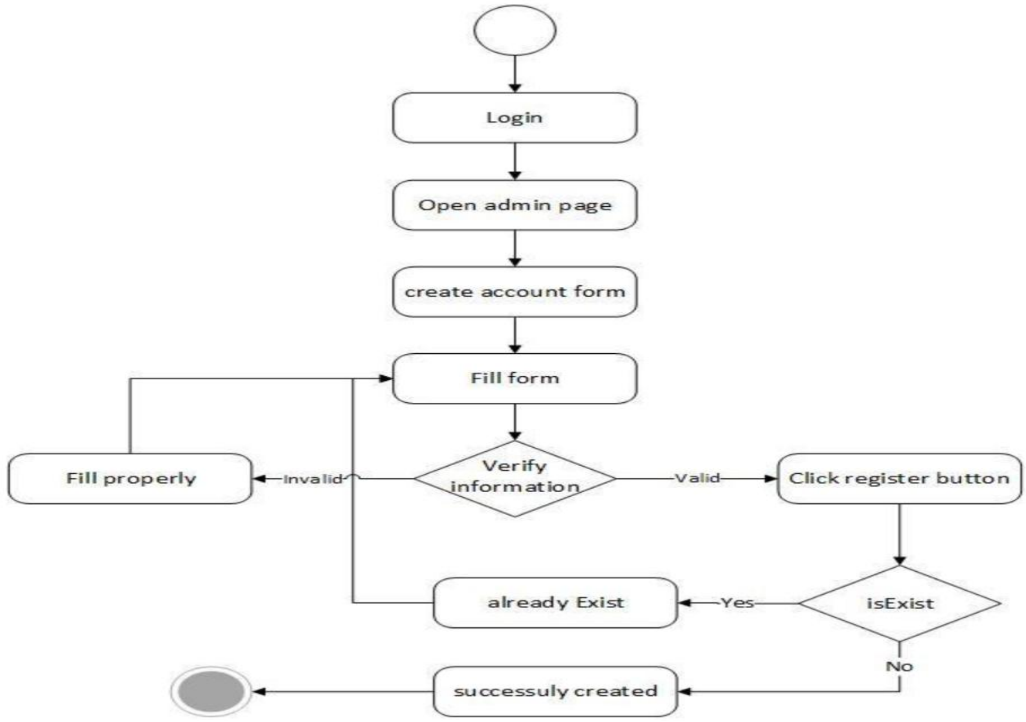


Figure 10:Create account activity diagram

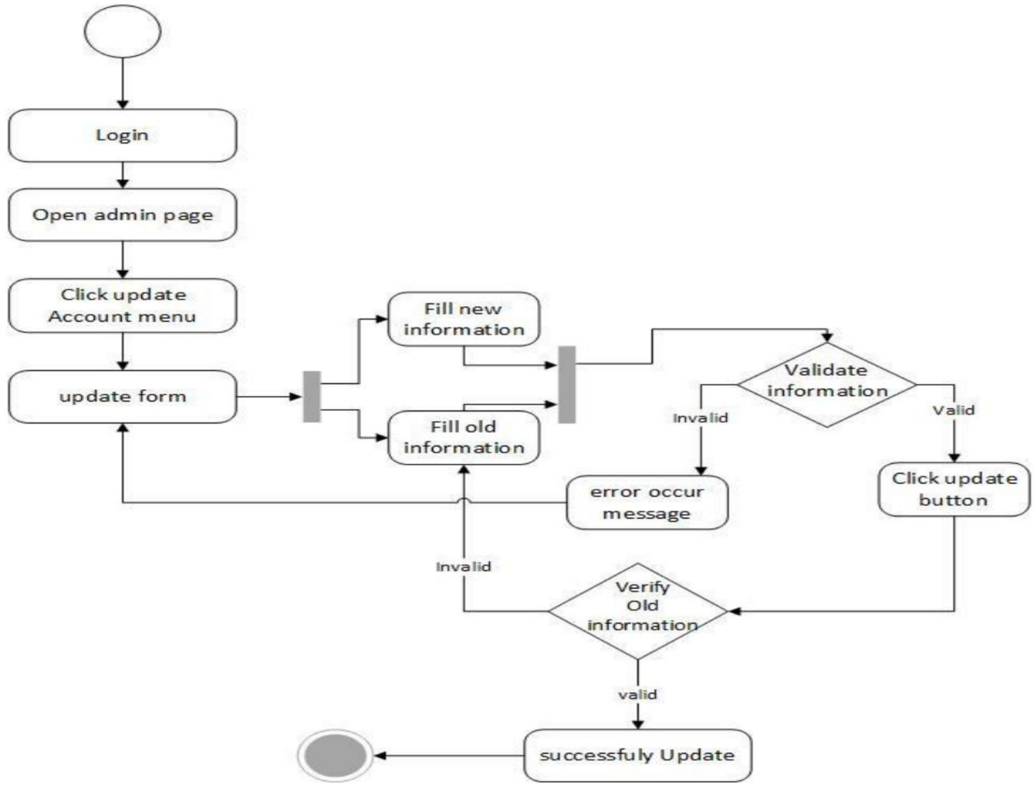


Figure 11:Update account activity diagram

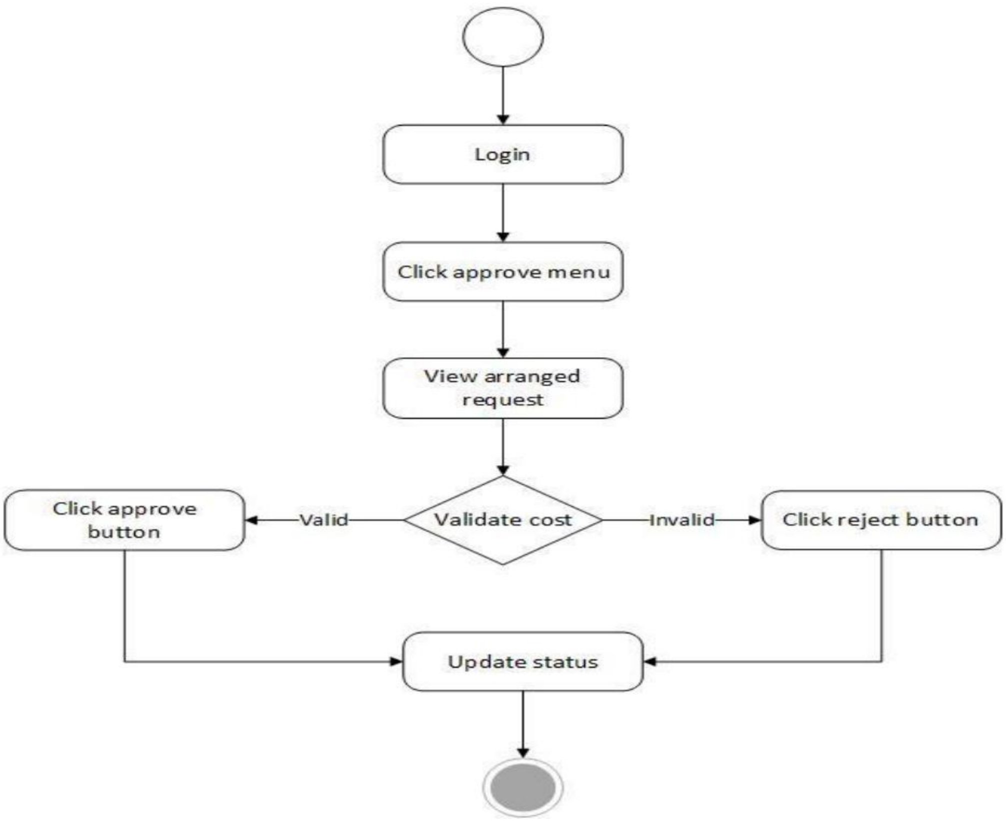


Figure 12:Approve request activity diagram

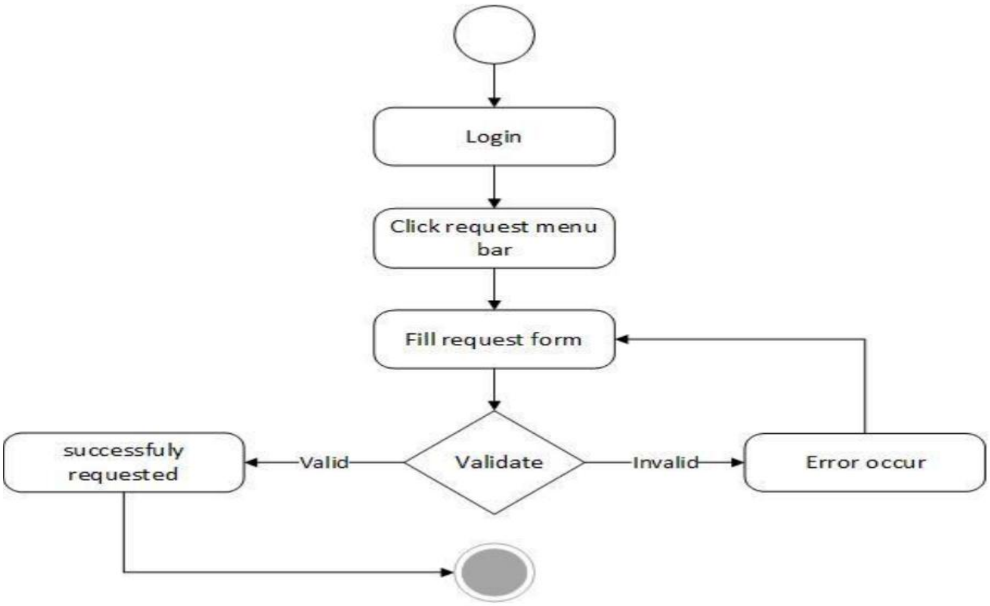


Figure 13:8 Request activity diagram

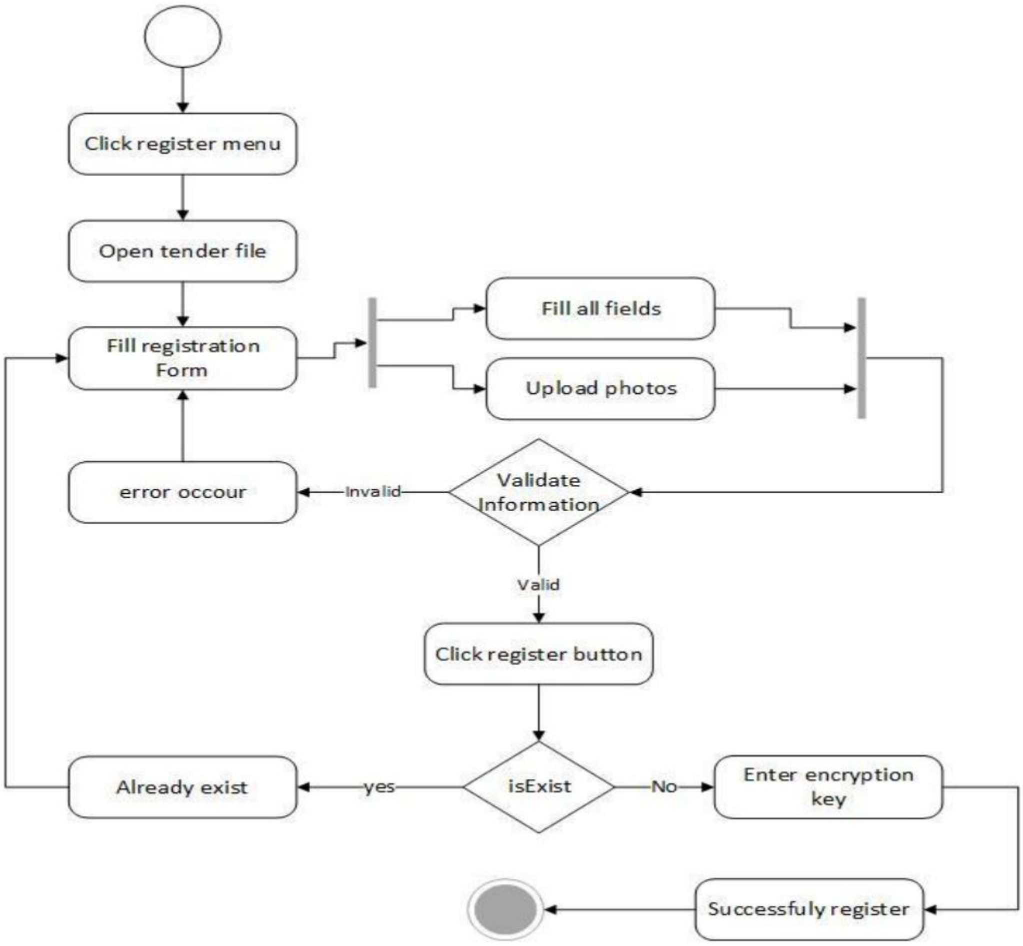


Figure 14:Register activity diagram

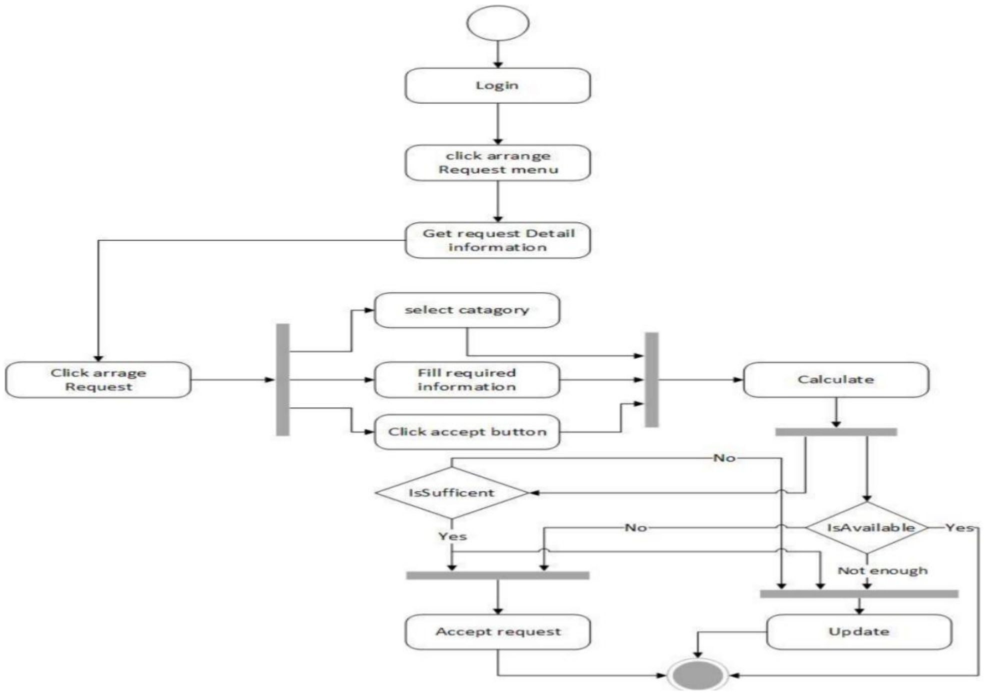


Figure 15:Arrange request activity diagram

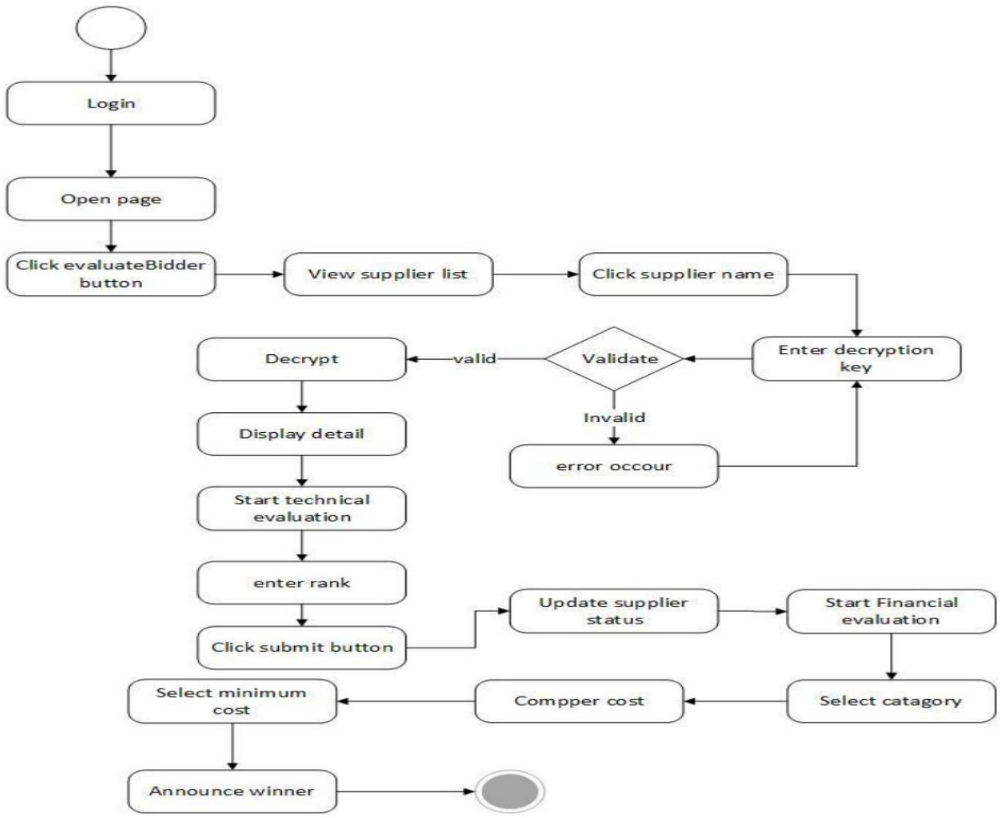


Figure 16:Evaluate bidder activity diagram

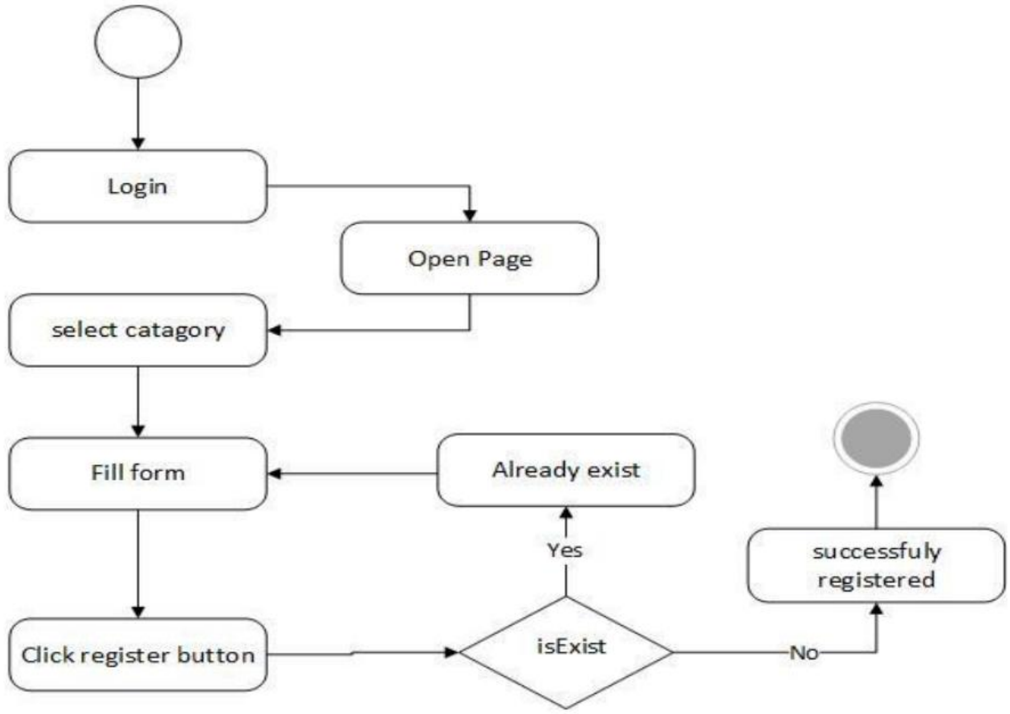


Figure 17:Register available data activity diagram

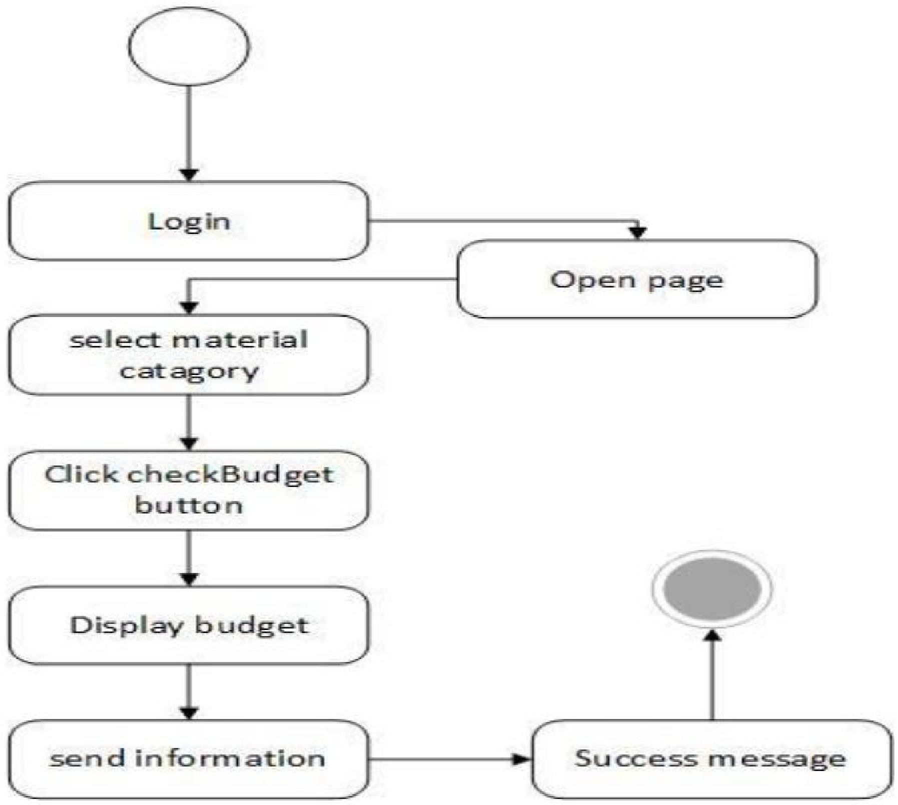


Figure 18:Check budget activity diagram

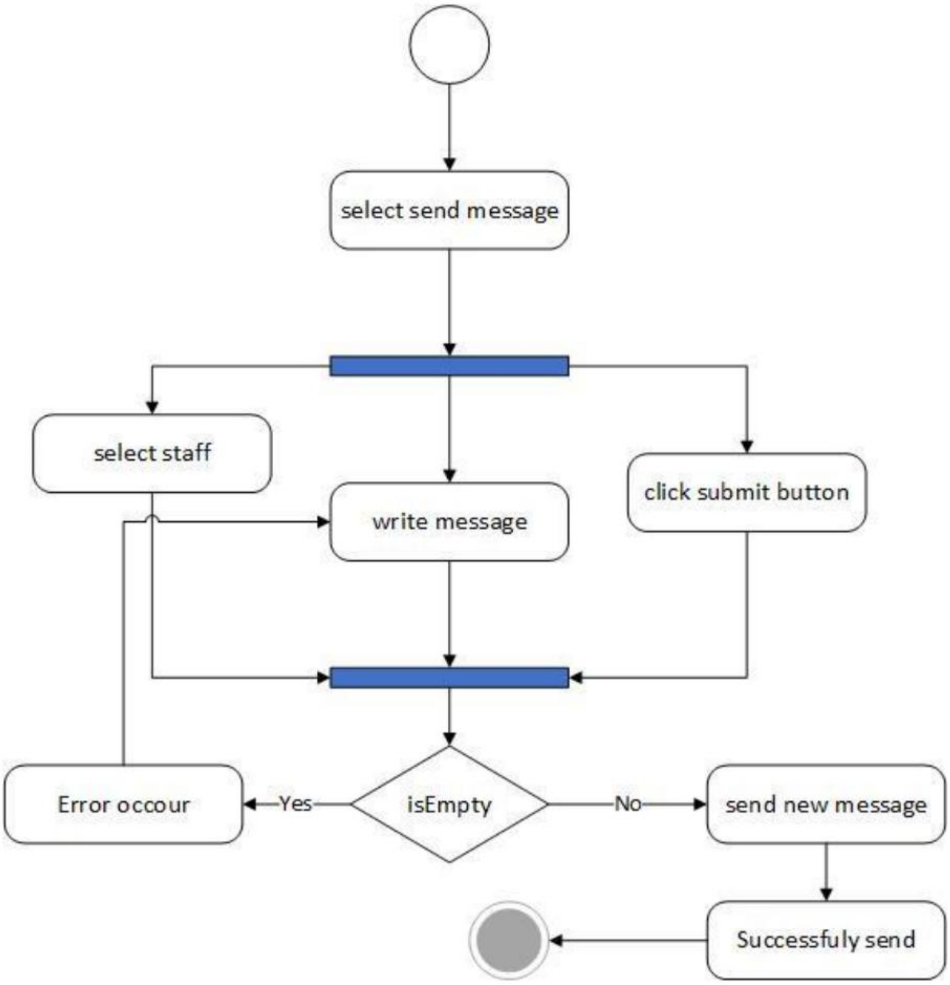


Figure 19:send message activity diagram

3.8. Class Diagram

A class diagram is an illustration of the relationships and source code dependencies among classes in the Unified Modeling Language (UML). In this context, a class defines the methods and variables in an object, which is a specific entity in a program or the unit of code representing that entity.

Generally, the project is including the following class in the class diagram the overview of the class diagram is:-

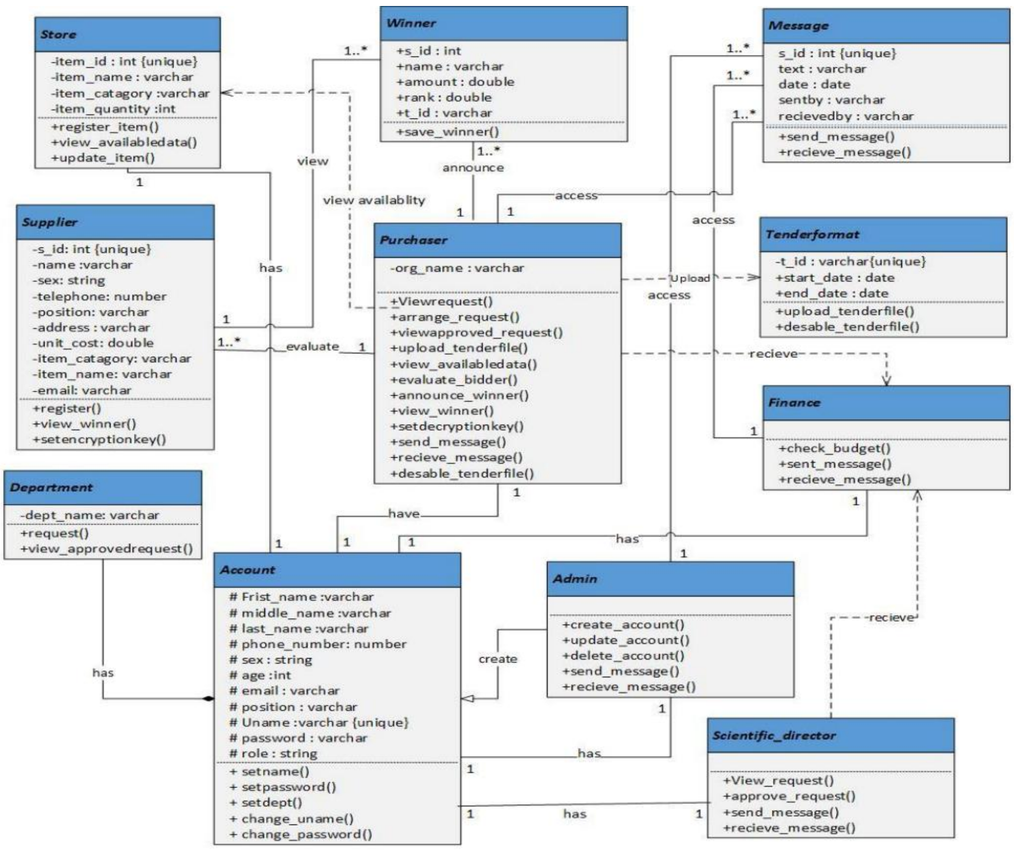


Figure 20: Class diagram

3.9. Non-Functional Requirements

The system must be accessible 24 hours a day, 7 days a week to accommodate suppliers who may be operating from different locations or submitting bids outside of standard office hours. Because procurement is a legal process, the system must ensure 100% data integrity, meaning that once a supplier submits a bid, it cannot be altered by any user, including the administrator, without leaving a verifiable audit trail.

To prevent corruption and "bid rigging," the system must implement Advanced Encryption Standards (AES) for all financial proposals; these files must remain locked and unreadable in the database until the exact "Bid Opening" timestamp occurs. Access to the system is strictly controlled through Role-Based Access Control (RBAC), ensuring that sensitive functions—like approving a purchase or viewing competitor prices—are restricted only to authorized personnel like the Scientific Director or the Purchasing Team.

From a performance standpoint, the system is required to handle at least 50 concurrent sessions without any degradation in service. Page load times should remain under 3 seconds on the university's internal network to ensure administrative efficiency. The user interface must be developed using responsive design principles (Bootstrap), allowing the university staff and external vendors to navigate the system seamlessly across desktops, laptops, and tablets without losing functionality.

Furthermore, the system must provide descriptive error handling to guide users—for example, preventing the upload of incorrect file formats or alerting a user if a mandatory field in a requisition form is missing. The backend must be designed using modular express and node.js structures, allowing for future scalability, such as integrating digital payment systems or expanding the system to other campuses of Debre Tabor University.

Finally, since the system will be hosted on-site, it must be capable of graceful recovery from sudden power failures or network drops, ensuring that no data is lost during a transaction. All labels, instructions, and system-generated reports must be presented clearly in English to match the official academic and technical standards of the IOT.

4. Chapter 4: System Design

4.1. Design Overview

The design of the E-Bid Processing System for DTU IOT centers on a full-stack JavaScript environment. By utilizing the MERN (MongoDB, Express.js, React, Node.js) stack, the design ensures high-speed data processing and a highly interactive user interface. This phase translates the procurement requirements into a modular, component-based blueprint. The primary goal is to maintain a seamless data flow between the React frontend and the Node.js backend using RESTful APIs, ensuring that all procurement activities from requisition to bid evaluation are handled in real-time with high security and scalability.

4.2. Chosen System Architecture

The system adopts a **3-Tier MERN Stack Architecture**. Unlike traditional multi-page applications, this architecture follows the **Single Page Application (SPA)** model, where the frontend and backend are decoupled and communicate exclusively via JSON data.

1. **Presentation Tier (React.js Frontend):**

   - This layer is built using **React.js** and **Vite**.
   - It manages the User Interface through a component-based structure, allowing for reusable UI elements (like bid cards, navigation bars, and status labels).
   - It handles client-side routing using **React Router**, ensuring a smooth, "no-refresh" experience for users.

2. **Application Tier (Node.js & Express.js Backend):**

   - **Node.js** serves as the asynchronous, event-driven runtime environment, which is highly efficient for handling concurrent users (like multiple suppliers bidding at once).
   - **Express.js** acts as the web framework to build **RESTful APIs**. It manages business logic, such as JWT-based authentication, role-based access control (RBAC), and tender deadline logic.

3. **Data Tier (MongoDB Database):**

   - **MongoDB** is used as the NoSQL document-oriented database.
   - Instead of rigid tables, it stores procurement data (Tenders, Bids, Users) as **JSON-like documents (BSON)**. This flexibility is ideal for handling technical specifications and varying bid documents without the need for complex SQL joins.
   - **Mongoose (ODM)** is used to provide a schema-based solution to model the application data.

4.3. User Interface Design

4.3.1. Description of the User Interface

The user interface for the E-Bid Processing System is built as a **Single Page Application (SPA)** using **React.js**. It is designed to be clean, professional, and efficient, ensuring that university staff and suppliers can navigate the procurement process with minimal training.

- **Dashboard-Driven Navigation:** Upon successful login, users are directed to a role-specific dashboard. The **Scientific Director** sees approval statistics, the **Purchasing Team** sees active tender counts, and **Suppliers** see upcoming bid deadlines.
- **Dynamic Data Tables:** The system utilizes interactive tables to display requisitions and bids. These tables include real-time filtering and sorting capabilities, allowing the Purchasing Team to quickly rank bidders.
- **Asynchronous Form Handling:** All forms (Registration, Requisition, and Bid Submission) are handled asynchronously using **Axios**. This allows for "soft" validation where error messages appear instantly without the user having to refresh the page.
- **Visual Status Indicators:** The UI uses color-coded badges to represent the lifecycle of a bid (e.g., **Pending**, **Approved**, **Rejected**, and **Closed**).
- **Responsive Layout:** Using **Tailwind CSS**, the interface is fully responsive. The sidebar collapses into a hamburger menu on smaller screens, ensuring the system is usable on tablets and mobile devices.

4.3.2. Screen shoot Images

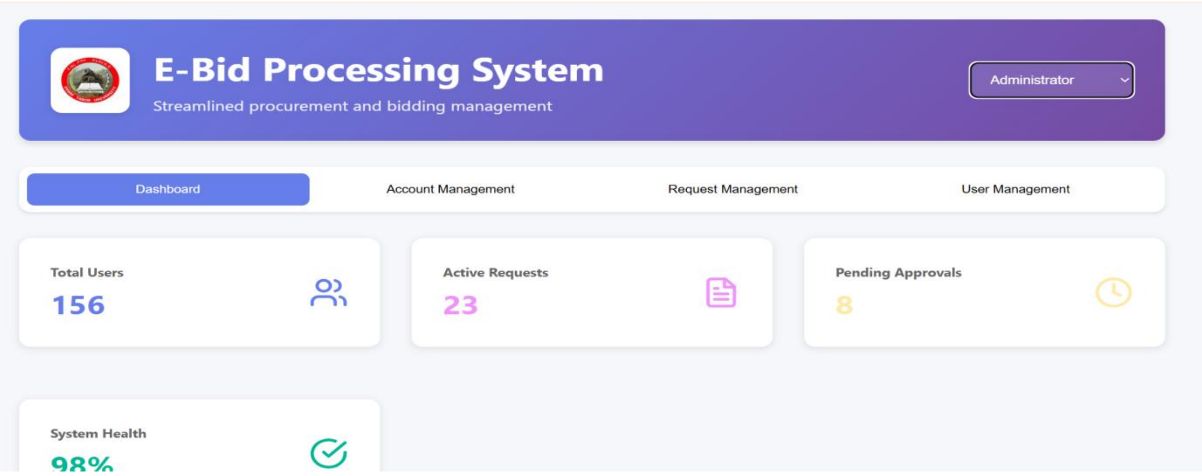


Image 1: System Admin Dashboard

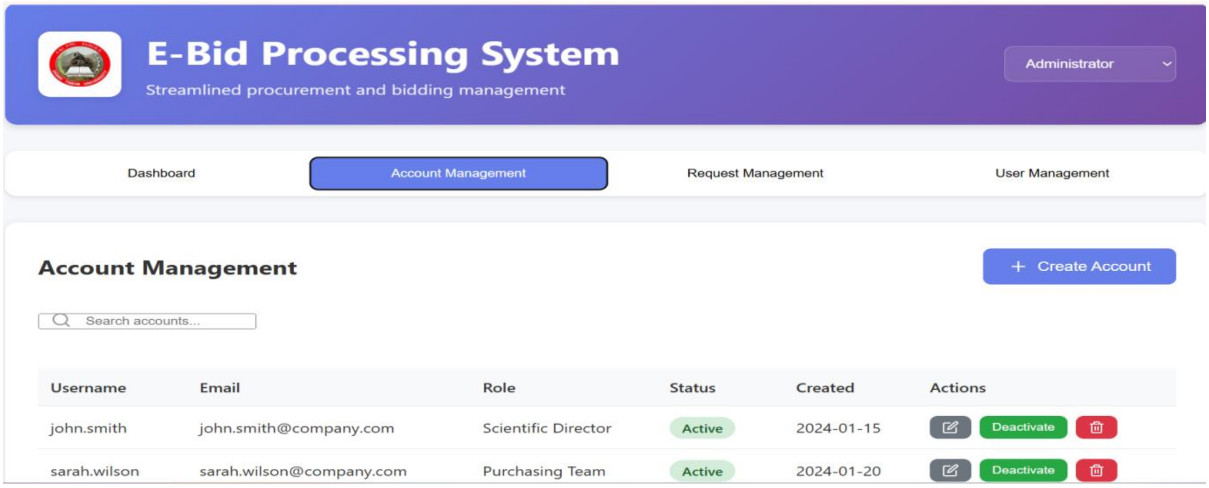


Image 2:Admin mange account

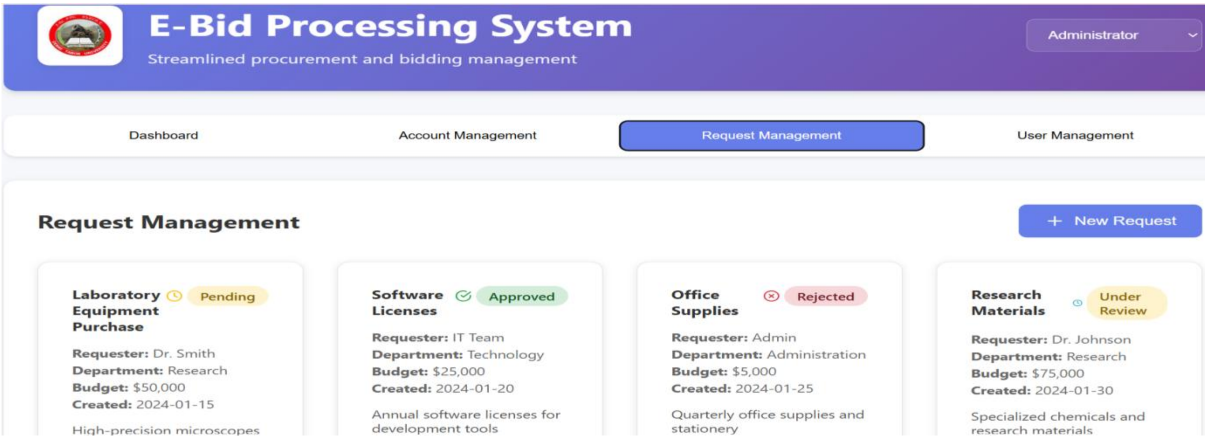


Image 3:Admin Request management

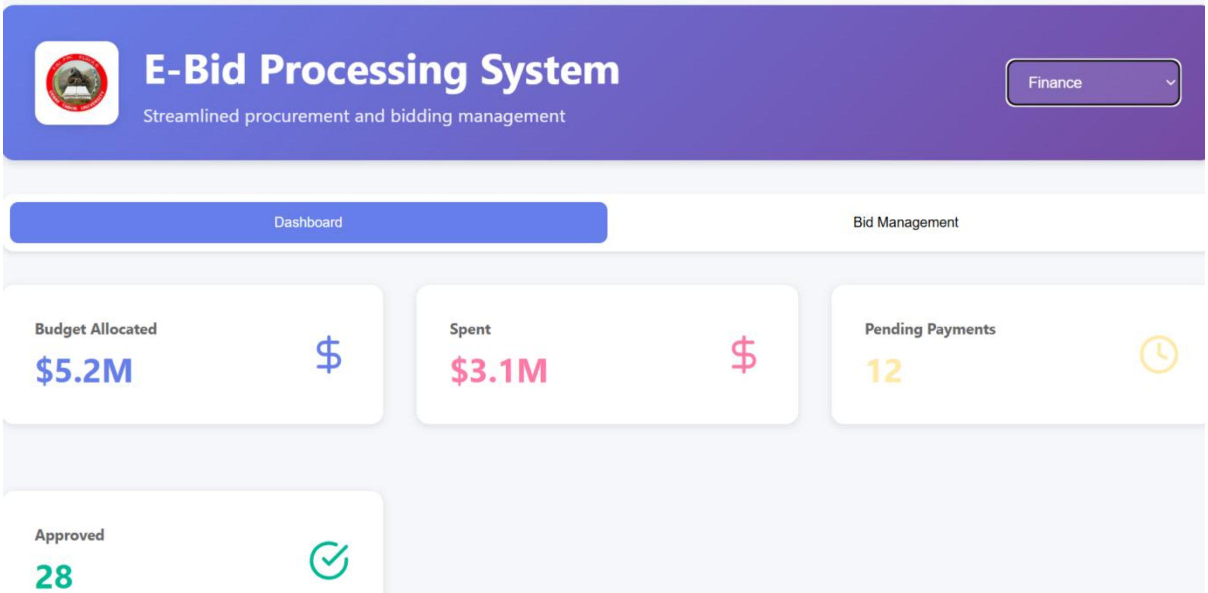


Image 4:Finance Dashboard

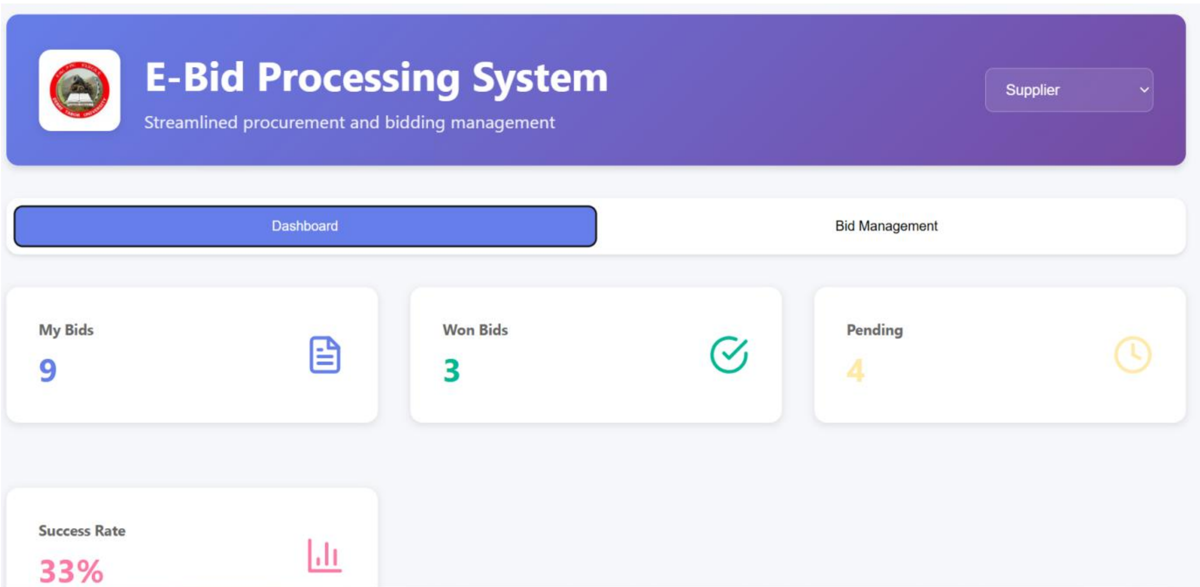


Image 5:Supplier Dashboard

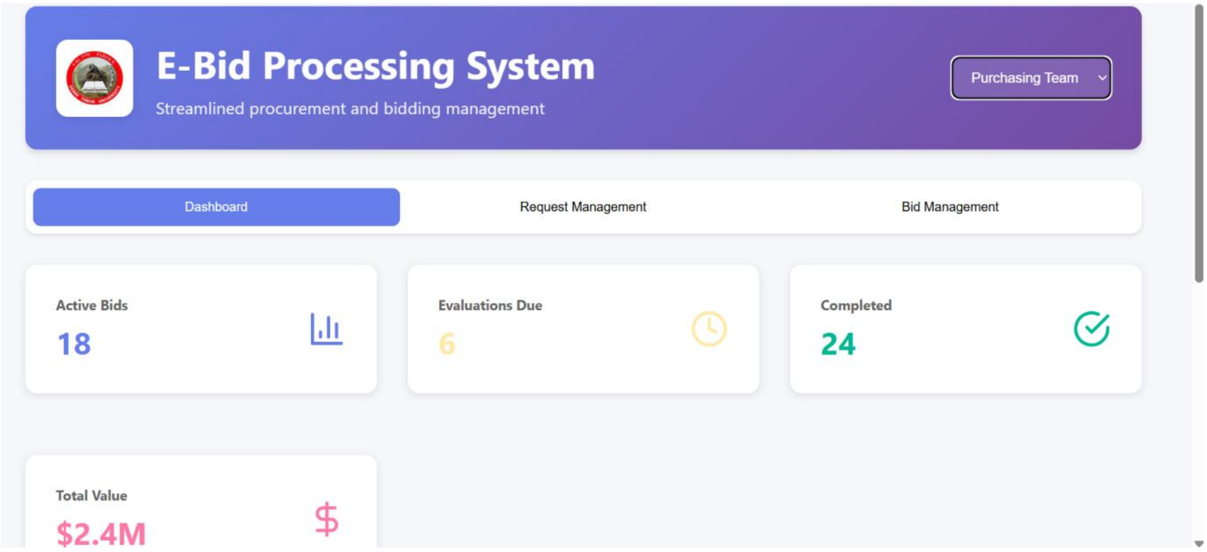
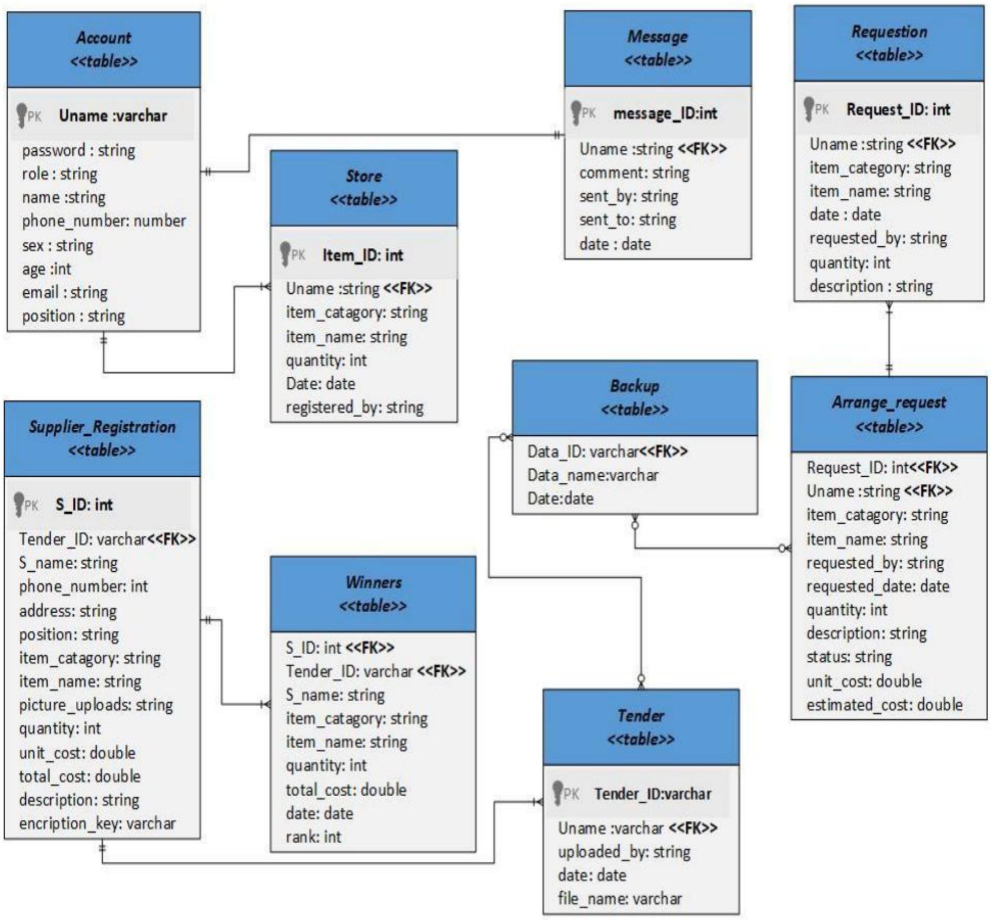


Image 6:Purchasing team Dashboard

4.4. Database Design

4.4.1. Entity-Relationship (ER) Diagram

The Entity-Relationship Diagram (ERD) represents the conceptual view of the data. Even in a NoSQL MERN stack, defining these relationships is vital for ensuring data consistency between users, tenders, and bids.



- ✓ **Entities:** User (Admin, Director, Supplier, etc.), Requisition, Tender, Bid, and Notification.
- ✓ **Relationships:User to Requisition:** One-to-Many (1:N) One department head can submit multiple requisitions.
- ✓ **Requisition to Tender:** One-to-One (1:1) One approved requisition leads to one tender announcement.
- ✓ **Tender to Bid:** One-to-Many (1:N) One tender can receive multiple bids from different suppliers.

✓ **User to Bid:** One-to-Many ($1:N$) — One supplier can bid on multiple different tenders.

4.4.2. Relational Data Model

In the MERN stack, the Relational Data Model is implemented via **Mongoose Schemas**. Below is the logical mapping of the primary collections used in the MongoDB database:

1. **Users Collection**

- _id (ObjectID, Primary Key)
- full_name (String)
- email (String, Unique)
- password (Hashed String)
- role (Enum: 'Admin', 'Director', 'Purchasing', 'Supplier')

2. **Tenders Collection**

- _id (ObjectID, Primary Key)
- title (String)
- description (String)
- closing_date (Date)
- created_by (ObjectID, Foreign Key referencing Users)
- status (String: 'Open', 'Closed', 'Evaluated')

3. **Bids Collection**

- _id (ObjectID, Primary Key)
- tender_id (ObjectID, Foreign Key referencing Tenders)
- supplier_id (ObjectID, Foreign Key referencing Users)
- financial_offer (Number, Encrypted)
- technical_doc_url (String/Link)
- submission_date (Date)

4.4.3. Normalization

1. **Requisition <<table>>**

Request_ID <<**PK**>>	Uname <<**FK**>>	Item_catagory	Item_name	Requested_by	date	quantity	description

1NF

Account <<table>>

Uname<<**PK**>>	Requester_Frist name	Requester_midle name	Requester_Last name

Requisition <<table>>

Request_ID <<**PK**>>	Uname <<**FK**>>	Item_name	Item_catagory	quantity	description	date

2NF

Requisition <<table>>

Request_ID <<**PK**>>	Date_ID<<**FK**>>	Item_name	Item_catagory	quantity	description

2. Message<<table>>

Message_ID <<**PK**>>	Comment	Sent_by	Sent_to	date	Uname <<**FK**>>

It fulfils ***1NF*** rule.

2NF Message <<table>>

Message_ID<<**PK**>>	Comment	Uname<<**FK**>>

Account <<table>>

Uname<<**PK**>>	Date_ID<<**FK**>>	Sender_F name	Sender_ Mname	Sent_to

Date <<table>>

Date_ID<<**PK**>>	Date

It fulfils ***3NF*** rule

3. Store<<table>>

Item_ID<<**PK**>>	Uname<<**FK**>>	Item_name	Item_catagory	Quantity	Date	Registered_buy

1NF

Store <<table>>

Item_ID<<**PK**>>	Uname<<**FK**>>	Item_name	Item_catagory	Quantity

Account <<table>>

Uname<<**PK**>>	Fname	Mname	Lname	Date

2NF

Date <<table>>

Date_ID<<**PK**>>	Item_ID<<**FK**>>	Date

It fulfills ***3NF*** rule.

4. Tender<<table>>

Tender_ID<<**PK**>>	Uname<<**FK**>>	Uploaded_by	Date	File_name

It fulfils ***1NF*** rule

2NF

Tender <<table>>

Tender_ID<<**PK**>>	Uname<<**FK**>>	File_name

Account <<table>>

Uname<<**PK**>>	Uploaded_by

Date <<table>>

Date_ID<<**PK**>>	Tender_ID<<**FK**>>	Date

It fulfils ***3NF*** rules

5. Winner<<table>>

S_ID <<**FK**>>	S_name	Tender_ID <<***FK***>>	Item_name	Item_catagory	quantity	Total_cost	Date	Rank

1NF

Supplier_registration <<table>>

S_ID <<**PK**>>>	S_name	Rank

Store <<table>>

Item_ID <<***PK***>>	S_ID <<**FK**>>	Tender_ID <<***FK***>>	Item_name	Item_catagory	quantity	Total_cost	Date

2NF

Store <<table>>

Item_ID<<***PK***>>	Tender_ID<<***FK***>>	Item_name	Item_catagory	quantity	Total_cost

Tender <<table>>

Tender_ID<<***PK***>>	S_ID <<**FK**>>	File_name

Date <<table>>

Date_ID<<**PK**>>	Tender_ID<<**FK**>>	Date

It fulfils ***3NF*** rules

5. Chapter 5: Testing

5.1. Test Plan

The test plan outlines the systematic approach to verifying that the E-Bid Processing System meets all functional and technical requirements. It ensures that the React frontend, Node.js backend, and MongoDB database interact correctly and securely.

5.1.1. Responsibilities of each group member

To ensure high-quality software, testing responsibilities are distributed among the team:

- **Lead Developer:** Responsible for **Unit Testing** of backend API routes and ensuring MongoDB schemas are validated correctly.
- **Frontend Developer:** Conducts **UI/UX Testing** to ensure React components render correctly across different screen sizes.
- **Quality Assurance (QA) Member:** Responsible for **System Integration Testing** and documenting bugs found during the process.
- **Security Lead:** Focuses on testing **JWT authentication** and role-based access controls to prevent unauthorized data access.

5.1.2. Types of Testing (load testing, Security testing, Performance testing etc.)

The following testing methodologies are applied to the MERN stack environment:

- **Unit Testing:** Testing individual functions and Express.js middleware to ensure they return the expected JSON responses.
- **Security Testing:** Verifying that passwords are encrypted (using Bcrypt) and that specific routes (like deleteTender) are only accessible to Authorized Admins.
- **Performance Testing:** Checking the system's ability to handle multiple concurrent bidders without significant increases in API response latency.
- **Load Testing:** Simulating a high volume of traffic (e.g., 50+ users) during the final hour of a bid deadline to ensure server stability.
- **Compatibility Testing:** Ensuring the React frontend works seamlessly on Google Chrome, Mozilla Firefox, and Safari.

5.1.3. Features to be Tested

- **Authentication & Authorization:** Success/failure of Login, Logout, and JWT token expiration.
- **Requisition Management:** The flow of a request from creation to approval by the Director.
- **Tender Posting:** Verification that new tenders appear in real-time on the Supplier dashboard.
- **Bid Submission:** Ensuring file uploads (PDFs) are stored correctly and financial data is encrypted.
- **Database Integrity:** Verifying that deleting a user does not leave "orphan" bid records in the MongoDB collection.

5.1.4. Features not to be Tested

- **Internet Connectivity:** We assume a stable university network environment; testing ISP-level failures is out of scope.
- **Third-Party Hosting:** Since the system is hosted locally for development, we will not test external cloud server (AWS/Azure) uptime.
- **Hardware Malfunction:** Failures of physical hardware like routers or server RAM are not part of the software test plan.

5.1.5. Testing Tools

- **Postman:** Used for testing and documenting the **Express.js REST APIs** without using the frontend.
- **Jest:** A JavaScript testing framework used for writing and running unit tests for Node.js logic.
- **React Testing Library:** Used to test React components to ensure they render and behave as expected.
- **JMeter:** An open-source tool used for **Load and Performance testing** of the backend server.
- **Chrome DevTools:** Used for debugging frontend layout, console errors, and network request monitoring.

5.2. Test Cases

The following table details the specific test scenarios used to validate the functionality, security, and usability of the E-Bid Processing System.

Scenario ID	Test Case Description	Priority	Action	Inputs	Expected Output	Actual Output	Test Browser	Result
TC-01	Verify User Login with valid credentials	High	Enter email and password; click Login	email: admin@dtu.edu.et, pass: Admin@123	Successful redirection to Admin Dashboard; JWT stored in LocalStorage.	As Expected	Chrome	Pass
TC-02	Verify User Login with invalid password	High	Enter correct email but wrong password	email: admin@dtu.edu.et, pass: Wrong123	Error message "Invalid Credentials" displayed; access denied.	As Expected	Chrome/ Firefox	Pass
TC-	Test	Hig	Supplier	URL:	System	As	Edge	Pas

Scenario ID	Test Case Description	Priority	Action	Inputs	Expected Output	Actual Output	Test Browser	Result
03	Role-Based Access Control (RBAC)	h	tries to access /admin/manage-users route	localhost:5173/admin/manage-users	redirects to 403 Unauthorized page or Home dashboard.	Expected		s
TC-04	Submit Purchase Requisition	Medium	Fill requisition form and click submit	Item: "Server RAM", Qty: 5, Dept: "ICT"	Data saved in MongoDB; Success toast message displayed.	As Expected	Chrome	Pass
TC-05	Financial Bid Submission (File Upload)	High	Supplier uploads PDF bid and enters amount	File: bid_prop.pdf, Amount: 50,000 ETB	PDF uploaded to server/storage; Bid record created	As Expected	Firefox	Pass

Scenario ID	Test Case Description	Priority	Action	Inputs	Expected Output	Actual Output	Test Browser	Result
					in database.			
TC-06	Bid Opening Date Constraint	Medium	Try to view bids before closing date	Click "View Bids" button on active tender	Button is disabled or "Bids are hidden until opening date" alert.	As Expected	Chrome	Pass
TC-07	Search and Filter Tenders	Low	Type keyword in search bar on Tender page	Search text: "Laboratory"	Table updates in real-time to show only Laboratory related tenders.	As Expected	Chrome	Pass
TC-08	Form Validation	Low	Click "Create Tender"	Empty inputs	Validation errors	As Expected	Safari	Pass

Scenario ID	**Test Case Description**	**Priority**	**Action**	**Inputs**	**Expected Output**	**Actual Output**	**Test Browser**	**Result**
	(Empty Fields)		without filling fields		appear below each required field.			

6. Chapter 6: Conclusion and Recommendation

6.1. Conclusion

The development of the E-Bid Processing System for Debre Tabor University's Institute of Technology successfully demonstrates how modern web technologies can transform traditional administrative workflows. By moving away from a manual, paper-based procurement process and adopting the **MERN (MongoDB, Express, React, Node.js) stack**, the system has achieved a higher level of transparency, efficiency, and data security.

The system successfully automates the lifecycle of a tender—from department requisitions and director approvals to supplier bidding and automated evaluation. Key objectives such as role-based access control, secure document handling, and real-time status tracking have been met. This digital transition not only reduces material wastage and human error but also ensures that the university's procurement activities are conducted in a fair and competitive environment, ultimately saving time and institutional resources.

6.2. Recommendation

Based on the development and testing phases of this project, the following recommendations are made to the DTU IOT management and ICT department:

- **User Training:** It is recommended that the university conduct short training workshops for department heads and the purchasing team to familiarize them with the React-based dashboard interface.
- **Infrastructure Stabilization:** To ensure 24/7 availability for external suppliers, the university should ensure the local server hosting the Node.js backend is supported by a robust backup power system (UPS) and a high-speed internet backbone.
- **Security Audits:** Periodic security audits should be performed to update the JWT (JSON Web Token) encryption keys and ensure the MongoDB database remains protected against unauthorized external access.
- **Supplier Awareness:** The university should encourage its regular vendors to register on the platform early to ensure a smooth transition from physical bid box submissions to digital uploads.

6.3. Future Work

While the current version of the E-Bid Processing System meets the core requirements, there is significant room for future expansion:

- **Payment Gateway Integration:** Future versions could integrate mobile money platforms (like Telebirr or M-Pesa) or CBE Birr to allow suppliers to pay for tender documents directly through the portal.
- **E-Signature Support:** Implementation of digital signatures to allow the Scientific Director to sign award letters and contracts electronically within the system.
- **SMS Notifications:** Beyond email alerts, integrating an SMS gateway to provide real-time bidding updates and deadline reminders to users on their mobile phones.
- **AI-Based Analytics:** Utilizing machine learning algorithms to analyze historical procurement data and predict market price trends for commonly requested laboratory equipment.
- **Multi-Campus Expansion:** Scaling the system architecture to support other campuses of Debre Tabor University beyond the Institute of Technology.

REFERENCE

1. **Ames, B.** (2023). *Full-Stack JavaScript Development with the MERN Stack*. O'Reilly Media.
2. **Banker, K., & Bakkum, P.** (2020). *MongoDB in Action: Covers MongoDB version 4.0*. Manning Publications.
3. **Chinnathotla, R. K.** (2021). *React and Node.js for Modern Web Applications*. Packt Publishing.
4. **Ethiopian Federal Government Procurement and Property Administration Agency (FPPA).** (2010). *Public Procurement Manual*. Addis Ababa, Ethiopia.
5. **Fielding, R. T.** (2000). *Architectural Styles and the Design of Network-based Software Architectures* (Doctoral dissertation, University of California, Irvine).
6. **Pressman, R. S., & Maxim, B. R.** (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.
7. **React Documentation.** (2026). *Managing State and Component Lifecycle*. Retrieved from https://react.dev.
8. **Sommerville, I.** (2016). *Software Engineering* (10th ed.). Pearson Education.

APPENDIX

Appendix A: System Implementation Code Snippets

To demonstrate the technical implementation of the MERN stack, key logic segments are provided below.

1. **Secure JWT Authentication Middleware (Node.js/Express)** This snippet shows how the system protects routes using JSON Web Tokens to ensure only authorized users (like the Scientific Director) can access specific actions.

```
const jwt = require('jsonwebtoken');

const protect = (req, res, next) => {

 let token = req.headers.authorization;

 if (token && token.startsWith('Bearer')) {

  try {

   const decoded = jwt.verify(token.split(' ')[1], process.env.JWT_SECRET);

   req.user = decoded;

   next();

  } catch (error) {

   res.status(401).json({ message: 'Not authorized, token failed' });

  }

 } else {

  res.status(401).json({ message: 'No token, authorization denied' });

 }

};
```

2. **Tender Schema Model (MongoDB/Mongoose)** This defines the data structure for a tender, ensuring all necessary procurement dates and statuses are captured.

```
const tenderSchema = new mongoose.Schema({
  tenderNumber: { type: String, unique: true },
  title: { type: String, required: true },
  description: String,
  openingDate: Date,
  closingDate: Date,
  budgetRange: Number,
  status: { type: String, enum: ['Draft', 'Published', 'Closed', 'Awarded'], default: 'Draft' }
}, { timestamps: true });
```

Appendix B: API Endpoints (Data Dictionary)

The following table serves as the developer's guide to the system's backend communication.

Endpoint	Method	Access	Description
/api/auth/login	POST	Public	Authenticates user and returns JWT.
/api/requisition/add	POST	Dept Head	Submits a new purchase request.
/api/tenders/all	GET	All Users	Retrieves list of active tenders.
/api/bids/submit	POST	Supplier	Uploads bid files and financial data.
/api/admin/approve/:id	PUT	Director	Updates requisition status to 'Approved'.